

ENTERPRISE KNOWLEDGE GRAPH PLATFORM

BodhiECG

Technical Documentation & Future Scope

This document describes the BodhiECG implementation as it exists in the codebase, together with the planned and candidate work that extends it. Statements about current behaviour are derived from modules under `api/`, `pipeline/`, `ingestion/`, `graph/`, `storage/`, `frontend/`, `docker/`, and `tests/`. Where a capability is not wired to the production API (`api.main:app`), that gap is stated explicitly. Section 24 is forward-looking and is not a commitment to delivery dates.

Unified edition — implementation reference and roadmap in a single document

Generated from repository source · 28 July 2026

Internal engineering document

Contents

1. Project Overview.....	5
1.1 Project Name and Purpose.....	5
1.2 Business Problem and Motivation.....	5
1.3 Objectives and Scope.....	5
1.4 Supported Data Sources and Document Types.....	5
1.5 Graph Operations and Current Capabilities.....	5
1.6 Current Limitations (from code).....	6
2. System Architecture.....	7
2.1 High-Level Topology.....	7
2.2 Subsystems.....	7
2.3 Ingestion Data Flow.....	7
2.4 Retrieval Data Flow.....	8
3. Repository Structure.....	9
3.1 Top-Level Map.....	9
3.2 Module Ownership Rule.....	9
4. Technology Stack.....	10
5. Application Flow.....	11
5.1 User Session Model.....	11
5.2 Typical Operator Flow.....	11
5.3 Pipeline Stage Semantics.....	11
6. Ingestion Pipeline.....	13
6.1 Engine: structured.....	13
6.2 Engine: template.....	13
6.3 Engine: unstructured.....	13
6.4 Engine: semi_structured.....	13
6.5 Error Handling and Confidence.....	13
7. Semantic Schema Generation.....	14
8. Knowledge Objects.....	15
9. Entity Resolution.....	16
10. Graph Construction.....	17
11. Graph Database.....	18
12. LLM Integration.....	19
13. Retrieval Pipeline.....	20
6A. Ingestion Engines — Implementation Detail.....	21
6A.1 Structured engine execution path.....	21
6A.2 Template engine execution path.....	21
6A.3 Unstructured engine execution path.....	21
6A.4 Semi-structured engine branches.....	21
6A.5 OCR pipeline (semi-structured API path).....	21
7A. Semantic Schema and Business Execution Schema.....	23
8A. Knowledge Object Lifecycle (ReviewStage).....	24
9A. Entity Resolution Strategies (EntityResolutionService).....	25

10A. Persistence and Indexing.....	26
12A. LLM Gateway Operations.....	27
21A. Docker Compose Services.....	28
16A. Storage Layout Under data/.....	29
14. API Documentation.....	30
14.1 Authentication.....	30
14.2 REST Endpoint Catalog.....	30
14.3 Example Request.....	33
15. Frontend.....	34
16. Database Schema.....	35
17. Configuration.....	36
18. Error Handling.....	37
19. Testing.....	38
20. Security.....	39
21. Deployment.....	40
22. Design Decisions.....	41
23. Known Limitations.....	42
24. Future Scope and Roadmap.....	43
24.1 Architecture-supported next steps.....	43
24.2 Near-term.....	43
24.3 Longer-term.....	44
24.4 Graph traversing algorithms for retrieval.....	45
24.5 Additional candidate scope (codebase-informed).....	47
24.6 How to use this section.....	47
24.7 Reference — roadmap themes (product).....	47
25. Appendices.....	49
25.1 Glossary.....	49
25.2 Sample Cypher.....	49
25.3 Related Repository Documents.....	49
Appendix A — Codebase Reference (imported).....	50
BodhiECG codebase reference.....	50
Appendix B — API Reference (imported).....	60
BodhiECG API reference.....	60
Appendix C — Run and Deployment Guide (from README.md).....	74
BodhiECG.....	74
Appendix D — Repository Directory Tree (partial).....	83
Appendix E — Frontend Module Inventory.....	103
Pages and features.....	103
API modules (frontend/src/api/).....	103
Routing.....	103
Appendix F — Pipeline Stage Source Files.....	104
Engine plugins.....	104
Appendix G — Structured Parser Classes.....	105

Appendix H — Test Suite Areas.....106

Appendix I — API Routers and Services Map.....107

1. Project Overview

1.1 Project Name and Purpose

BodhiECG (package name `bodhiectg` in `pyproject.toml`) is a unified enterprise knowledge graph platform. Per `README.md`, it ingests structured CSVs, unstructured documents, and semi-structured scans (invoices, medical bills) into a FalkorDB graph per workspace, then supports natural-language querying via a structured NL→Cypher retrieval orchestrator and grounded GraphRAG.

1.2 Business Problem and Motivation

Organizations hold operational data in tabular files, prose documents, and scanned forms. BodhiECG consolidates these into a single graph model with shared pipeline stages (review, entity resolution, persistence, validation) so downstream consumption (exploration, NL query, alignment review) operates on one workspace-bound graph (`Workspace.graph_name` in FalkorDB).

1.3 Objectives and Scope

- Single production API entry: `uvicorn api.main:app` (not legacy ingestion/`*/api` apps).
- Single ingestion orchestrator: `pipeline.orchestrator.PipelineOrchestrator` with engine plugins.
- Four engines: structured, unstructured, template, `semi_structured` (`pipeline/registry.py`).
- JSON workspace metadata under `data/`; graph persistence via `graph.falkordb.FalkorGraphClient`.
- React frontend for workspace, ingestion, pipeline monitoring, review, graph, retrieval, alignment.

1.4 Supported Data Sources and Document Types

`InputClassifier` (`pipeline/dispatcher.py`) defines extension families:

Table 1: Classification extension families

Category	Extensions / rules	Engine
Structured tabular	<code>.csv</code> , <code>.xlsx</code> , <code>.xls</code> , <code>.json</code> , <code>.sql</code>	structured (unless template routing)
Template tabular	<code>template_id</code> or OWL + tabular source	template
Semi-structured scans	<code>.pdf</code> and images if templates/ <code>*.json</code> exist	<code>semi_structured</code>
Unstructured documents	<code>.pdf</code> , <code>.docx</code> , <code>.pptx</code> , images, <code>.txt</code> , <code>.md</code> (when not semi route)	unstructured

1.5 Graph Operations and Current Capabilities

- Workspace CRUD, stats, clear-graph (`api/routers/workspaces.py`).
- Graph snapshot, metrics, node, neighborhood, path, export (`api/routers/graph.py`).

- `POST /query` — 16-stage RetrievalOrchestrator (`ingestion/structured/retrieval/orchestration/orchestrator.py`).
- `POST /retrieval/grounded` — grounded GraphRAG (`ingestion/unstructured/graph/context.py` via `retrieval_service`).
- Semi-structured template OCR, cache queues, batch graph ingest (`api/routers/semi_structured.py`).

1.6 Current Limitations (from code)

- No end-user login UI; JWT auth exists (`api/auth/jwt_auth.py`) but is disabled when `JWT_SECRET` is unset.
- Production routes do not depend on `require_auth` by default — open API when JWT unset.
- `KnowledgeAlignmentStage` applies to `semi_structured` generic branch, not all engines.
- `ingestion/unstructured/graph/retrieval_substrate.py` exists; not mounted as a dedicated API reindex route.
- Legacy parallel paths: `ingestion/structured/api`, `ingestion/unstructured/orchestration` — bypass unified pipeline.
- Root packages `entity_resolution/`, `ontology/`, `semantic_schema/` are placeholders; logic lives in `pipeline/` and `ingestion/`.
- Redis settings (`config/settings.py` `RedisSettings`) default `enabled=false` — not a production cache layer today.

2. System Architecture

2.1 High-Level Topology

```

Browser
├─ Docker :8080 → nginx (frontend) —/api/*→ FastAPI :8000 → FalkorDB :6379
├─ Local :5173 → Vite proxy /api → uvicorn api.main:app → FalkorDB

```

Figure 1: Deployment topology (from `README.md`)

2.2 Subsystems

Table 2: Major subsystems

Subsystem	Implementation	Role
Frontend	frontend/ React + Vite + TypeScript	UI; proxies <code>/api</code> to backend
Backend	api/ FastAPI	HTTP façade; services call orchestrator and stores
Pipeline	pipeline/	Unified ingest stages + engine registry
Graph DB	FalkorDB via falkordb Python client	Per-workspace graphs
LLM layer	LLM gateway HTTP (config <code>LLMGatewaySettings</code>)	Chat, embeddings, vision — not hard-coded OpenAI endpoint
Storage	storage/*.py + data/	Workspaces JSON, alignment, entity registry, history
Knowledge objects	models.domain.SemanticObject	Intermediate representation → graph builder
Entity resolution	<code>pipeline/stages/entity_resolution.py</code> + structured ER service / unstructured linker	Dedup and cross-doc identity
Validation	<code>pipeline/stages/validation.py</code>	Post-persist structural/semantic checks
Retrieval	ingestion/structured/retrieval/* + <code>api/services/retrieval_service.py</code>	Query answering
Graph builder	<code>graph_builder/canonical.py</code> CanonicalGraphBuilder	SemanticObject → nodes/edges

2.3 Ingestion Data Flow

HTTP or CLI (`app.py`)

→ `PipelineOrchestrator.DEFAULT_STAGES`

→ `ClassificationStage` → `IngestionStage` (engine plugin)

→ SemanticSchemaStage → KnowledgeObjectStage → ReviewStage
→ EntityResolutionStage → KnowledgeAlignmentStage → GraphBuilderStage
→ PersistenceStage → ValidationStage → IndexingStage → RetrievalReadyStage

Figure 2: Unified pipeline stage order (`pipeline/orchestrator.py`)

2.4 Retrieval Data Flow

`POST /query` invokes `RetrievalOrchestrator` with 16 injected stages: query understanding, workspace/graph selection, ontology context, schema introspection, entity linking, query planning, Cypher generation, graph retrieval, expansion, hybrid retrieval, ranking, context build, answer generation, citation validation, metrics (see docstring in `ingestion/structured/retrieval/orchestration/orchestrator.py`).

`POST /retrieval/grounded` uses unstructured graph context assembly (entity vectors, MENTIONS passages) for cited prose answers — separate code path from the 16-stage orchestrator.

3. Repository Structure

3.1 Top-Level Map

See [docs/CODEBASE.md](#) §2. Key directories:

Table 3: Top-level directories

Path	Responsibility
api/	Production FastAPI — routers, services, schemas only
pipeline/	Orchestrator, stages, engines, dispatcher
ingestion/	structured, unstructured, semi_structured , template engines
graph/	FalkorGraphClient, mock fallback, Cypher normalize
graph_builder/	CanonicalGraphBuilder
storage/	WorkspaceStore, entity registry, retrieval history
models/	domain.py shared Pydantic types
config/	settings.py — sole env reader
common/	logging, exceptions, canonical stage names
llm/	Gateway client wrapper
frontend/	React UI
docker/	Backend, frontend, FalkorDB Dockerfiles
tests/	Pytest mirror of packages
docs/	CODEBASE.md , API_REFERENCE.md , this DOCX output
templates/	Semi-structured JSON templates
data/	Runtime state (workspaces, uploads, caches)
app.py	Streamlit CLI — same orchestrator, not production UI

3.2 Module Ownership Rule

api/routers must not import pipeline or graph directly; api/services/* is the only layer that orchestrates pipeline.*, ingestion.*, graph.*, and storage.* (documented in [api/main.py](#) and [docs/CODEBASE.md](#) §3).

4. Technology Stack

Table 4: Technology stack (from dependencies and imports)

Technology	Where used	Role
Python 3.12+	<code>pyproject.toml</code> requires-python	Runtime for API, pipeline, ingestion
FastAPI + Uvicorn	<code>api/main.py</code>	Production HTTP server
Pydantic v2	<code>models/</code> , <code>api/schemas/</code>	Validation and API contracts
React + TypeScript + Vite	<code>frontend/</code>	SPA; dev server port 5173
Tailwind CSS v4	<code>frontend/vite.config.ts</code>	UI styling via <code>@tailwindcss/vite</code>
FalkorDB	<code>graph/falkordb.py</code>	Graph database (Redis protocol, port 6379)
pandas, openpyxl	structured parsers	CSV/Excel ingestion
Docling, pypdf, python-docx, python-pptx	ingestion parsers	Document layout and formats
Tesseract / EasyOCR / Docling	<code>ingestion/semi_structured/ocr.py</code>	OCR cascade for scans
spaCy <code>en_core_web_sm</code>	<code>pyproject.toml</code>	Structured NER leg of HybridNEREngine
GLiNER / GLiNER2 / Relex	<code>ingestion/unstructured/extraction/</code>	Unstructured NER+relations
owlready2, rdflib	ontology parsing	OWL for template engine
LangGraph	<code>semi_structured</code> and template orchestration	Branching ingest graphs
langchain-openai, openai SDK	<code>pyproject.toml</code>	HTTP clients — typically pointed at LLM gateway, not direct vendor requirement
NetworkX	<code>ingestion/structured/graph/analytics.py</code>	Optional graph analytics (lazy import)
RapidFuzz, splink, duckdb	entity resolution / unstructured	Fuzzy match and linking utilities
pytest, httpx	<code>tests/</code>	Automated tests
Docker Compose	<code>docker-compose.yml</code>	FalkorDB + backend + frontend stack

WHY gateway abstraction: `config/settings.py` centralizes `LLM_GATEWAY_*` so corporate deployments use one OpenAI-compatible endpoint (JWT/token) rather than embedding vendor URLs in ingestion code.

5. Application Flow

5.1 User Session Model

There is no login page in `frontend/src/routes/index.tsx`. Users open the app at `/workspace`. Optional JWT (JWT_SECRET) can gate routes that explicitly Depends(`require_auth`) — production routers currently do not enforce JWT globally.

5.2 Typical Operator Flow

- Create or select workspace (WorkspacePage → GET/POST `/workspaces`).
- Upload files (POST `/upload`) or use semi-structured ingest (POST `/semi-structured/ingest`).
- Start pipeline (POST `/pipeline/start`) with IngestRequest; watch SSE GET `/pipeline/{run_id}/stream`.
- Inspect knowledge objects and review (ReviewPage, `/review/*`, `/knowledge-objects`).
- Alignment review for semi-structured generic path (AlignmentPage, `/alignment/*`).
- Explore graph (GraphPage, CytoscapeCanvas, `/graph/*`).
- Query (RetrievalPage, POST `/query` or `/retrieval/grounded`).

5.3 Pipeline Stage Semantics

Table 5: DEFAULT_STAGES (`pipeline/orchestrator.py`)

Stage	Purpose
WorkspaceStage	Attach workspace and graph client
ClassificationStage	InputClassifier → <code>selected_engine</code>
IngestionStage	Engine plugin run() — parse/extract only
SemanticSchemaStage	LLM/schema bridge when applicable (may no-op)
KnowledgeObjectStage	Build SemanticObject list
ReviewStage	Auto-approve above threshold; HITL statuses
EntityResolutionStage	ER service or cross-doc linker or deterministic passthrough
KnowledgeAlignmentStage	Semi-structured generic merge decisions
GraphBuilderStage	CanonicalGraphBuilder
PersistenceStage	FalkorDB MERGE; chunks/MENTIONS for unstructured
ValidationStage	Structural + semantic validation
IndexingStage	Embeddings HNSW/BM25 — unstructured engine path

Stage	Purpose
RetrievalReadyStage	Marks retrieval metadata ready

6. Ingestion Pipeline

6.1 Engine: structured

`pipeline/engines/structured_engine.py` delegates to structured parsers (`ingestion/structured/parsers/`), chunking (`chunking/`), and HybridNEREngine (`ner/`). Produces entities and relations.

6.2 Engine: template

`pipeline/engines/template_engine.py` runs LangGraph CSV+OWL pipeline (`ingestion/structured/orchestration/langgraph_pipeline.py`). Deterministic primary keys and concept nodes.

6.3 Engine: unstructured

`pipeline/engines/unstructured_engine.py` calls `ingestion/unstructured/knowledge_object/pipeline.py`: Docling parse (`cocoindex_flow/parse_fn.py`), GLiNER-Relex extraction, cleaning, optional coref, resolution merge.

6.4 Engine: semi_structured

`pipeline/engines/semi_structured_engine.py` and `ingestion/semi_structured/orchestration/langgraph_pipeline.py` branch: template path (`signature + template_extractor.py`) vs generic KV/LLM schema (`kv_extraction.py`, `business_schema.py`).

6.5 Error Handling and Confidence

PipelineContext accumulates warnings and errors (`pipeline/context.py`). API returns them in `ApiResponse.warnings` and inside `IngestResponse`. Confidence is modeled as `models.domain.Confidence` on entities and `SemanticObjects`.

7. Semantic Schema Generation

`pipeline/stages/semantic_schema.py` implements `SemanticSchemaStage`. Structured flows include execution schema and semantic schema builders under `ingestion/structured/` (`tests/structured/unit/test_semantic_schema.py`, `test_execution_schema.py`). Semi-structured generic branch uses LLM business schema generation (`ingestion/semi_structured/business_schema.py`). Template engine maps columns via OWL ontology parser.

8. Knowledge Objects

Product term Knowledge Object maps to `models.domain.SemanticObject` (see class docstring in `models/domain.py`). Fields include `canonical_id`, `entity`, `entity_type`, `properties`, `relationships`, `confidence`, `provenance`, `ontology_mapping`, `validation_status` (approved | needs_review | rejected), and structured provenance columns (`source_table`, `primary_key`, `foreign_keys`) for CSV/template flows.

API: GET/PATCH `/knowledge-objects` scoped by `run_id` (in-memory RunRecord). Human review also via `/review/*` and ReviewStage auto-approval thresholds.

9. Entity Resolution

EntityResolutionStage ([pipeline/stages/entity_resolution.py](#)): template and deterministic semi-structured identities passthrough; unstructured attempts cross-doc linker then EntityResolutionService ([ingestion/structured/entity_resolution/service.py](#)) with six strategies: exact, case-insensitive, normalization, ontology-alias, fuzzy (RapidFuzz), embedding.

10. Graph Construction

GraphBuilderStage uses `graph_builder/canonical.py` CanonicalGraphBuilder to emit GraphNode and GraphEdge from non-rejected SemanticObjects. PersistenceStage writes via FalkorGraphClient MERGE operations (`graph/falkordb.py`), including chunk nodes and MENTIONS edges for unstructured content.

11. Graph Database

FalkorDB connection settings: `config/settings.py` FalkorDBSettings (FALKORDB_HOST, PORT, `graph_name`). Each workspace has `graph_name` in `storage/workspace_store.py`. Mock client (`graph/mock.py`) when FALKORDB_USE MOCK or connection failure paths in tests.

Frontend graph visualization: `frontend/src/features/graph/CytoscapeCanvas.tsx` on GraphPage.

12. LLM Integration

Configuration: LLMGatewaySettings in `config/settings.py` — `base_url`, `token/jwt`, model aliases, `embed_model`, `vision_model`, `timeout`, `max_retries`, `daily_budget`.

Clients: `llm/client.py`; `ingestion/unstructured/llm/gateway.py` for embeddings/chat/vision; structured retrieval uses gateway clients under `ingestion/structured/`. Health check: `uv run python -m ingestion.unstructured.llm.healthcheck`.

The `openai` Python package is used as an HTTP client compatible with the gateway; deployments are not required to call OpenAI directly if the gateway fronts other models (`.env.example` shows `claude/gemini` model names).

13. Retrieval Pipeline

Two production modes wired in `api/routers/retrieval.py`:

- Structured: RetrievalOrchestrator 16 stages → `POST /query`, streaming variant `POST /retrieval/query/stream`.
- Grounded GraphRAG: `POST /retrieval/grounded` with optional query rewrite (`GroundedQueryRequest.rewrite`).
- NL2Cypher: `POST /retrieval/nl2cypher` with execute flag and self-correction attempts in response.
- Supporting: `/retrieval/plan`, `/retrieval/context`, `/retrieval/explain`, `/cypher`, `/cypher/validate`, `/retrieval/history`.

Table 6: RetrievalOrchestrator stages (docstring)

#	Retrieval stage component
1	IQueryUnderstanding
2	IWorkspaceSelector
3	IGraphSelector
4	IOntologyContextLoader
5	ISchemaIntrospector
6	IEntityLinker
7	IQueryPlanner
8	ICypherGenerator
9	IGraphRetriever
10	IGraphExpander
11	IHybridRetriever (vector/BM25 where configured)
12	IRanker
13	IContextBuilder
14	IAnswerGenerator
15	ICitationValidator
16	IRetrievalMetricsCollector

6A. Ingestion Engines — Implementation Detail

6A.1 Structured engine execution path

StructuredEngine (`pipeline/engines/structured_engine.py`) loads the source through `ingestion/structured/parsers/get_parser(extension)`. Parsers inherit `BaseParser` (`parsers/base.py`) and return normalized tabular or text artifacts consumed by chunking (`ChunkerOrchestrator` in `chunking/`) and `HybridNEREngine` (`ner/`). Output entities are assembled into `SemanticObjects` in `KnowledgeObjectStage` via engine-specific builders under `ingestion/structured/knowledge_object/`.

6A.2 Template engine execution path

TemplateEngine requires CSV paths plus OWL ontology (`IngestRequest.ontology_path` or `ontology_id`). It executes `ingestion/structured/orchestration/langgraph_pipeline.py` which performs deterministic join discovery, concept column detection, and relationship extraction aligned to OWL classes. Canonical IDs follow table:primary_key and Concept:column:value patterns documented in `pipeline/stages/entity_resolution.py` (deterministic passthrough).

6A.3 Unstructured engine execution path

- Parse: `ingestion/unstructured/cocoindex_flow/parse_fn.py` (Docling; optional vision via `LLM_GATEWAY_VISION_MODEL`).
- Chunk: hierarchical chunking aligned with extraction windows.
- Extract: `ingestion/unstructured/extraction/relex.py` — GLiNER-Relex joint entity and relation extraction.
- Clean: `extraction/clean.py` removes boilerplate and merges fragmented spans.
- Coref (optional): `extraction/coref.py` fastcoref when enabled.
- Resolve: `resolution/merge.py`, `scoring.py`, `linker.py` for within-document identity.
- Ontology labels: `ingestion/unstructured/ontology/loader.py` reads YAML POLE configs; optional LLM generate via API.

6A.4 Semi-structured engine branches

SemiStructuredEngine (`pipeline/engines/semi_structured_engine.py`) runs `ingestion/semi_structured/orchestration/langgraph_pipeline.py`. When a document matches `templates/*.json` signature, `template_extractor.py` performs deterministic field extraction and `template_ko_builder.py` writes KOs with `EntityRegistryStore` business keys. The generic branch uses `kv_extraction.py` and `business_schema.py` (LLM semantic schema) and then `KnowledgeAlignmentStage` with `pipeline/alignment/knowledge_alignment.py` hybrid scoring (exact, fuzzy, ontology, embedding, LLM signals).

6A.5 OCR pipeline (semi-structured API path)

`ingestion/semi_structured/ocr.py` implements Tesseract via `pytesseract` for images, `pypdf` text layer for PDFs, Docling fallback when character count below `SEMI_PDF_FAST_MIN_CHARS`, optional EasyOCR when

OCR_FALLBACK enabled. Environment variables documented in .env.example (TESSERACT_CMD, OCR_UPSCALE, OCR_FALLBACK).

7A. Semantic Schema and Business Execution Schema

SemanticSchemaStage ([pipeline/stages/semantic_schema.py](#)) bridges engine-native outputs to unified semantic representations. Structured code includes execution schema builders ([tests/structured/unit/test_execution_schema.py](#)) and semantic schema tests ([test_semantic_schema.py](#)). Semi-structured generic branch generates business schema via LLM in [business_schema.py](#). Template path maps columns to OWL via ontology parser modules under [ingestion/structured/knowledge_object/](#) and [ingestion/structured/ontology/](#).

8A. Knowledge Object Lifecycle (ReviewStage)

ReviewStage applies `AUTO_APPROVE_THRESHOLD` from `ingestion/structured/knowledge_object/human_review` — confidence score ≥ 0.8 auto-approves via `review_status()`; otherwise `validation_status` remains `needs_review`. Unattended API runs `record_review_decisions` on `PipelineContext` but do not pause the pipeline; `GraphBuilderStage` includes non-rejected objects per `graph_builder` rules. Frontend `ReviewPage` and `PATCH /knowledge-objects` allow human correction; `/review/label-action` supports bulk approve/reject/remove by `entity_type`.

9A. Entity Resolution Strategies (EntityResolutionService)

When EntityResolutionService runs (structured/unstructured paths after cross-doc attempt), it chains six strategies documented in `pipeline/stages/entity_resolution.py`: exact match, case-insensitive, normalization, ontology-alias, fuzzy (RapidFuzz), embedding similarity. Template and deterministic semi-structured identities skip fuzzy matching to prevent collapsing distinct primary keys.

10A. Persistence and Indexing

PersistenceStage ([pipeline/stages/persistence.py](#)) uses FalkorGraphClient to MERGE nodes and relationships. For unstructured runs it persists text chunks and MENTIONS edges linking entities to source passages. IndexingStage ([pipeline/stages/indexing.py](#)) runs for unstructured engine: creates embedding vectors and HNSW vector index plus BM25 full-text index ([ingestion/unstructured/graph/indexes.py](#)) when EMBED_ON_INGEST and gateway embed model are configured.

12A. LLM Gateway Operations

- Chat completions — template authoring, business schema, alignment LLM signal, answer generation.
- Embeddings — entity/chunk vectors for indexing and grounded retrieval.
- Vision — optional figure description during Docling when DESCRIBE_FIGURES=1.
- Healthcheck module — [ingestion/unstructured/llm/healthcheck.py](#) validates connectivity.
- Retries — LLMGatewaySettings.max_retries (default 3); timeout LLM_GATEWAY_TIMEOUT_SECONDS.

21A. Docker Compose Services

`docker-compose.yml` defines three application services plus volumes (verify file for exact service names and ports):

- `falkordb` — graph database; ports 6379 and 3000 (browser UI in FalkorDB image).
- `backend` — builds `docker/backend/Dockerfile`; unicorn `api.main:app`; bind-mounts `./data` and `./templates`.
- `frontend` — nginx static assets; proxies `/api` to backend; exposed port 8080.

Backend entrypoint `docker/backend/entrypoint.sh` ensures data directory structure at container start. HF cache may be bind-mounted under `./data/huggingface` for model reuse (see compose comments).

16A. Storage Layout Under data/

- `data/workspaces/workspaces.json` — workspace registry (WorkspaceStore).
- `data/workspaces/knowledge_objects/` — persisted KO snapshots per workspace when written.
- `data/workspaces/semantic_memory/` — alignment semantic memory payloads.
- `data/workspaces/alignment_decisions/` — alignment audit JSON.
- `data/semi_structured/cache/` — `pending.json`, `needs_review.json`, `ingested.json` document records.
- `data/semi_structured/uploads/` — staged scan images.
- Upload paths under workspace-specific dirs from `ingestion_service.save_upload`.

14. API Documentation

Production app: `api/main.py`. Response envelope: `api/schemas/common.py` ApiResponse — status, message, data, errors, warnings.

14.1 Authentication

JWT optional via `api/auth/jwt_auth.require_auth`. When `JWT_SECRET` is empty, auth is a no-op. LLM gateway uses separate `LLM_GATEWAY_JWT/TOKEN` — not the same as API JWT.

14.2 REST Endpoint Catalog

Table 7: Production REST endpoints (`api/routers/`*)

Method	Path	Summary	Body	HTTP
GET	<code>/health</code>	System liveness	None	200
GET	<code>/workspaces</code>	List workspaces	—	200
POST	<code>/workspaces</code>	Create workspace	WorkspaceCreateRequest	201
GET	<code>/workspaces/{id}</code>	Get workspace	—	200/404
PATCH	<code>/workspaces/{id}</code>	Rename	WorkspaceRenameRequest	200/400
DELETE	<code>/workspaces/{id}</code>	Delete workspace	—	200/404
GET	<code>/workspaces/{id}/stats</code>	Stats + active run	—	200
POST	<code>/workspaces/{id}/clear-graph</code>	Wipe graph nodes/edges	—	200
GET	<code>/data/csv</code>	List CSV files	—	200
GET	<code>/data/documents</code>	List documents	—	200
GET	<code>/data/ontologies</code>	List OWL files	—	200
GET	<code>/data/ontologies/preview</code>	OWL preview	query path	200/404
GET	<code>/data/ontologies/unstructured</code>	List YAML ontologies	—	200
POST	<code>/data/ontologies/unstructured/generate</code>	LLM YAML ontology	GenerateOntologyRequest	200
POST	<code>/upload</code>	Stage upload	multipart file	200
POST	<code>/ingest/detect</code>	Engine preview	DetectEngineRequest	200
POST	<code>/ingest</code>	Sync ingest	IngestRequest	200
POST	<code>/pipeline/start</code>	Async ingest	IngestRequest	202

Method	Path	Summary	Body	HTTP
GET	/pipeline/{run_id}	Run summary	—	200
GET	/pipeline/{run_id}/status	Status	—	200
GET	/pipeline/{run_id}/logs	Logs	—	200
GET	/pipeline/{run_id}/artifacts	Artifacts	—	200
GET	/pipeline/{run_id}/metrics	Metrics	—	200
GET	/pipeline/{run_id}/stage/{stage}	Stage detail	—	200
POST	/pipeline/{run_id}/resume	Resume paused run	—	200/400
GET	/pipeline/{run_id}/stream	SSE progress	—	200 stream
GET	/knowledge-objects	List KOs	query run_id	200
GET	/knowledge-objects/{id}	Get KO	query run_id	200
PATCH	/knowledge-objects/{id}	Patch KO	KnowledgeObjectPatchRequest	200
GET	/review/concept-columns	Concept columns	run_id	200
POST	/review/concept-mode	Toggle concept/property	ConceptModeRequest	200
GET	/review/labels	Label summaries	run_id	200
POST	/review/label-action	Bulk label action	LabelActionRequest	200
GET	/graph/list	All FalkorDB graphs	—	200
POST	/graph/register	Register graph	RegisterGraphRequest	201
GET	/graph	Snapshot	workspace_id	200
GET	/graph/metrics	Graph metrics	workspace_id	200
GET	/graph/node/{id}	Node detail	workspace_id	200
GET	/graph/neighborhood/{id}	Neighborhood	workspace_id	200
POST	/graph/path	Shortest path	PathRequest	200

Method	Path	Summary	Body	HTTP
POST	/graph/export	Export graph	GraphExportRequest	200
POST	/query	16-stage retrieval	QueryRequest	200
POST	/retrieval/grounded	GraphRAG	GroundedQueryRequest	200
POST	/retrieval/nl2cypher	NL→Cypher	Nl2CypherRequest	200
POST	/retrieval/plan	Retrieval plan	RetrievalPlanRequest	200
POST	/retrieval/context	Context only	QueryRequest	200
POST	/retrieval/explain	Explainability	RetrievalExplainRequest	200
POST	/retrieval/query/stream	SSE retrieval	QueryRequest	stream
GET	/retrieval/history	Query history	optional workspace_id	200
POST	/cypher/validate	Validate Cypher	CypherValidateRequest	200
POST	/cypher	Execute Cypher	CypherRequest	200/400
GET	/alignment/{ws}/semantic-memory	Semantic memory	—	200/404
GET	/alignment/{ws}/decisions	Alignment decisions	—	200
POST	/alignment/override	HITL override	AlignmentOverrideRequest	200
GET	/semi-structured/templates	List templates	—	200
GET	/semi-structured/templates/{domain}	Template JSON	—	200
POST	/semi-structured/templates/author	Author template	AuthorTemplateRequest	200
POST	/semi-structured/templates/draft	Draft template	multipart	200
POST	/semi-structured/templates/save	Save template	SaveTemplateRequest	200
POST	/semi-structured/ingest	OCR ingest	multipart	200
POST	/semi-structured/ingest/path	Ingest by path	query params	200
GET	/semi-structured/documents	Document cache	status filter	200

Method	Path	Summary	Body	HTTP
GET	/semi-structured/status	Cache stats	—	200
GET	/semi-structured/workspace/{id}/entities	Merged entities	—	200
POST	/semi-structured/documents/{id}/resolve	Review resolve	—	200
POST	/semi-structured/documents/{id}/mark-ingested	Mark ingested	—	200
POST	/semi-structured/documents/reprocess	Batch reprocess	status query	200
POST	/semi-structured/documents/ingest-pending	Graph pending batch	query params	200
POST	/semi-structured/documents/{id}/reprocess	Single reprocess	—	200
POST	/semi-structured/ontology/export	Export OWL	—	200
POST	/semi-structured/export/csv	Export CSV	—	200

14.3 Example Request

POST http://localhost:8000/pipeline/start

Content-Type: application/json

```
{
  "source": "data/uploads/example.csv",
  "workspace_id": "ws_abc123",
  "engine_hint": null
}
```

Figure 3: Async pipeline start example

Via Docker UI proxy: POST http://localhost:8080/api/pipeline/start with the same JSON body.

15. Frontend

Table 8: Frontend routes (`frontend/src/routes/index.tsx`)

Route	Page	Purpose
<code>/workspace</code>	WorkspacePage	Workspace selection and management
<code>/ingestion</code>	IngestionPage	Upload, templates, scanned docs tab
<code>/pipeline/:runId?</code>	PipelinePage	Run status, SSE stream
<code>/knowledge-objects</code>	KnowledgeObjectsPage	KO browser
<code>/review</code>	ReviewPage	Concept columns and label actions
<code>/alignment</code>	AlignmentPage	Alignment decisions and overrides
<code>/graph</code>	GraphPage	Cytoscape graph explorer
<code>/retrieval</code>	RetrievalPage	Query / grounded / Cypher UI

API client: `frontend/src/api/client.ts` and feature modules. State: React Query patterns in hooks (see `frontend/src/hooks/`). Layout: AppLayout, Sidebar, Topbar. DevPanel for JSON debugging.

16. Database Schema

FalkorDB is schema-flexible; labels and relationship types are driven by ingestion outputs, not a fixed SQL DDL. Workspace isolation: each workspace maps to a `graph_name`. Semi-structured template ontologies declare entity classes and business keys (`ingestion/semi_structured/template_ko_builder.py`, `EntityRegistryStore`).

Unstructured indexing creates vector indexes (HNSW) and BM25 full-text indexes in `ingestion/unstructured/graph/indexes.py` when `IndexingStage` runs.

There is no single checked-in Cypher constraints catalog — inspect live graph via `GET /graph/metrics` or FalkorDB Browser :3000.

17. Configuration

All environment variables load through `config/settings.py get_settings()`. Template: `.env.example`.

- `FALKORDB_*` — graph connection
- `LLM_GATEWAY_*` — models, JWT, embeddings, vision
- `API_*` — host, port, CORS
- `JWT_*` — optional API auth
- `TESSERACT_CMD`, `OCR_*`, `SEMI_PDF_*` — semi-structured OCR
- `ONTOLOGY_PATH`, `EMBED_ON_INGEST`, `EMBED_DIM` — unstructured ontology and indexing
- `STORAGE_*` — workspace store paths

18. Error Handling

- `api/middleware/error_middleware.py` — maps exceptions to API responses
- `common/exceptions.py` — `NotFoundError`, `ValidationError` used in services
- Pipeline non-fatal warnings via `context.add_warning`; unstructured cross-doc link failures fall through
- LLM gateway `max_retries` in `LLMGatewaySettings`
- Semi-structured validation → `needs_review` status in document cache

19. Testing

Run: `uv run pytest` (see `tests/README.md` if present). ~180 test modules under `tests/api`, `tests/pipeline`, `tests/structured` (unit, integration, `cross_domain`, e2e), `tests/unstructured`, `tests/semi_structured`, `tests/config`, `tests/stabilization`.

Coverage is area-specific — no single coverage gate documented in `pyproject.toml` from default read.

20. Security

- Secrets in .env (not committed); pydantic SecretStr for tokens
- Cypher validation before `POST /cypher` execution
- Multipart upload size limits — default FastAPI/starlette behavior
- Prompt injection: no dedicated middleware documented; retrieval passes user query to LLM gateway
- Input validation via Pydantic on all API bodies

21. Deployment

`docker-compose.yml`: services falkordb, backend, frontend. Backend Dockerfile:

`docker/backend/Dockerfile` (multi-stage Python 3.12, uv, tesseract). Frontend: nginx serves static build and proxies `/api`. Registry override: `docker-compose.registry.yml` for pre-built images. `data/` and `templates/` bind-mounted.

22. Design Decisions

Table 9: Major design decisions

Decision	Choice	Benefit	Tradeoff
Unified orchestrator	One DEFAULT_STAGES list for all engines	Avoids duplicate persist/validate paths in legacy ingestion APIs	Tradeoff: legacy apps still exist and can confuse operators
SemanticObject as KO	Single IR type in <code>models/domain.py</code>	Graph builder stays engine-agnostic	Structured-only fields optional on unstructured sources
FalkorDB	Redis-protocol graph DB with Cypher	Single container in compose; fits MERGE-heavy ingest	Not a relational source-of-truth for tabular staging
LLM gateway	Corporate JWT gateway instead of inline API keys in code	Centralized model routing and billing	Requires gateway availability for NL features

23. Known Limitations

- No multi-user RBAC or login UI in frontend.
- RunRegistry holds pipeline state in process memory — restarts lose in-flight run handles.
- JSON .sql sources supported as structured; prose JSON may classify as structured not unstructured unless `engine_hint`.
- Graphiti-core listed in dependencies — verify active production path before relying on temporal memory.
- CocolIndex flow (`ingestion/unstructured/cocolindex_flow/flow.py`) is incremental batch — not the unified API path.

24. Future Scope and Roadmap

This section describes **planned and candidate** work for BodhiECG. It is **not** a promise of delivery dates or committed backlog items. Capabilities listed here are either explicitly on the product roadmap (team references), naturally suggested by gaps in the current codebase ([docs/CODEBASE.md](#)), or extensions of retrieval/graph behavior that fit the existing architecture.

Today's production surface remains `api.main:app`, the unified `PipelineOrchestrator`, and the two major retrieval paths documented in [docs/API_REFERENCE.md](#): structured 16-stage NL→Cypher (`POST /query`) and grounded GraphRAG (`POST /retrieval/grounded`).

24.1 Architecture-supported next steps

These items require no new architecture — every hook already exists in the codebase.

- Wire `retrieval_substrate.ensure_retrieval_substrate` for non-unstructured grounded retrieval backfill.
- Mount `require_auth` on routers when `JWT_SECRET` is configured, for production hardening.
- Unify `BaseEntityResolver` to select structured ER vs unstructured linker per workspace.
- Deprecate legacy ingestion REST apps in favor of `api.main` only.
- Enable `RedisSettings` when job queue caching is implemented.

24.2 Near-term

These items align with the “**Where we’re going — near-term**” direction: improve NL→Cypher reliability on real schemas, simplify retrieval for operators, and strengthen evaluation beyond a single domain.

Table 12: Near-term initiatives

Initiative	Rationale	Fit with current code
Preserve integer primary keys + fix junction-table counting	Weak or many-to-many schemas often produce Cypher that miscounts or loses join semantics when PK types are coerced to strings or when junction tables are treated as entity tables.	Structured ingestion and template engine already emit table/PK metadata on <code>SemanticObject</code> (<code>source_table</code> , <code>primary_key</code> , <code>foreign_keys</code> in <code>models/domain.py</code>). NL→Cypher stages (<code>ingestion/structured/retrieval/cypher_generation/</code> , schema introspection) can use typed PK hints and explicit junction-node patterns in prompts and validators.
Unify retrieval into one router	Operators today choose between <code>/query</code> (orchestrator pipeline), <code>/retrieval/grounded</code> (GraphRAG), <code>/retrieval/nl2cypher</code> , <code>plan/context/explain</code> , and streaming	<code>api/routers/retrieval.py</code> already groups routes; a single entry (e.g. <code>POST /retrieval</code> with <code>mode: structured grounded nl2cypher</code>) would delegate to

Initiative	Rationale	Fit with current code
	variants.	<code>api/services/retrieval_service.py</code> without changing underlying orchestrators.
Complete Spider 1.0 benchmark run + expand GraphRAG-Bench	Text-to-SQL/graph benchmarks stress schema linking and grounding quality across domains.	Spider-style evaluation is not wired as a CI job in this repo today; ontology auto-generation notes GraphRAG-Bench motivation in <code>ingestion/unstructured/ontology/generate.py</code> . Future work: scripted eval harness under <code>tests/</code> or <code>notebooks/</code> , reusable against workspace graphs.
Wire grounded-retrieval substrate for all engines	<code>ingestion/unstructured/graph/retrieval_substrate.py</code> backfills entity embeddings and rebuilds HNSW/BM25 when graphs were built without unstructured indexing.	Indexing today is strongest on the unstructured ingest path; template/structured runs may lack vectors until substrate runs. Near-term: call <code>ensure_retrieval_substrate()</code> from <code>IndexingStage</code> and/or lazy path on <code>POST /retrieval/grounded</code> (module docstring describes intent).
Production auth and API hardening	JWT exists but is off when <code>JWT_SECRET</code> is unset; routes do not globally depend on <code>require_auth</code> .	<code>api/auth/jwt_auth.py</code> + <code>config/settings.py</code> JWTSettings — enable per-environment and attach to routers when accounts exist.
Deprecate legacy duplicate APIs	<code>ingestion/structured/api</code> , unstructured direct-write paths, and Streamlit <code>app.py</code> bypass the unified orchestrator for some flows.	Document and remove from operator runbooks; single entry remains <code>uvicorn api.main:app</code> .

24.3 Longer-term

Direction from “Where we’re going — longer-term” plus architecture extensions that reuse workspace graphs, KOs, and HITL audit trails.

Table 13: Longer-term initiatives

Initiative	Description	Architectural hook
Measure layer (Observation nodes / SQL)	Heterogeneous quantitative facts (labs, KPIs, time series) often do not map cleanly to entity–entity relations alone. An Observation (or Measurement) node layer — or federated SQL over source systems — would sit beside entity graphs.	<code>SemanticObject</code> / graph builder could add observation labels; FalkorDB remains primary store or SQL is queried at retrieval time and fused in context builder (stage 13 of retrieval orchestrator).

Initiative	Description	Architectural hook
Self-healing curator agent	An agent monitors the graph for redundant edges, wrong merges, or ontology violations and proposes or applies fixes, with audit log.	Builds on <code>ValidationStage</code> , alignment decisions JSON under <code>data/workspaces/alignment_decisions/</code> , and future write-safe Cypher with human approval.
Cross-pipeline ontology reconciliation + active learning	YAML (unstructured GLiNER), OWL (template), and LLM-generated semi-structured schemas can disagree on labels. Reconciliation maps synonyms to workspace canonical vocabulary; active learning uses HITL from <code>ReviewStage</code> and <code>/alignment/override</code> .	GET <code>/alignment/{workspace_id}/semantic-memory/pipeline/alignment/knowledge_alignment.py</code> , unstructured <code>ontology/resolver.py</code> .
Unified entity-resolution strategy selection	Today: deterministic passthrough (template/semi template), unstructured cross-doc linker, then structured <code>EntityResolutionService</code> six-strategy chain.	<code>pipeline/stages/entity_resolution.py</code> docstring recommends a future <code>BaseEntityResolver</code> facade per workspace policy.
Redis-backed job queue and run durability	<code>RunRegistry</code> is in-process; restarts lose live run handles. <code>RedisSettings</code> defaults to <code>enabled=false</code> .	Enable Redis for pipeline job state, SSE fan-out across workers, and cached schema snapshots for retrieval.
Temporal / episodic memory	<code>graphiti-core[falkordb]</code> is a dependency; confirm production wiring before relying on it.	Optional layer for time-stamped facts and conflict resolution across ingestions.

24.4 Graph traversing algorithms for retrieval

24.4.1 What exists today

Structured retrieval already includes **graph traversal as a first-class concept**, not only single-hop Cypher results:

Table 14: Traversal components in the current codebase

Component	Location	Behavior
Query intent	<code>ingestion/structured/retrieval/query_understanding/</code>	Intents include <code>TRAVERSAL</code> , <code>GRAPH_TRAVERSAL</code> , <code>MULTI_HOP</code> (<code>retrieval/models.py</code>).
Query planning	<code>query_planning/planner.py</code>	Sets <code>max_hops</code> (1-3) from intent and config.
Graph expansion (Stage 10)	<code>graph_expansion/expander.py</code>	Neighbour expansion over N hops with token budget; optional community and ontology hierarchy expansion; confidence-based

Component	Location	Behavior
		stopping.
Grounded GraphRAG	ingestion/unstructured/graph/context.py	Builds a typed subgraph from entity-vector seeds and MENTIONS-linked passages — traversal is implicit in subgraph assembly, not a separate algorithm library.
Graph explorer API	GET /graph/neighborhood/{node_id}?hops=	Ego-network for UI (CytoscapeCanvas), separate from retrieval orchestrator.

This is primarily **breadth-first neighbour expansion in Cypher** plus budget caps, not a pluggable library of classical graph algorithms.

24.4.2 Future scope — traversal and ranking

The following are **candidate enhancements** that fit FalkorDB + the existing [IGraphExpander](#) / [IGraphRetriever](#) interfaces without replacing the 16-stage orchestrator:

1. **Weighted and typed walks.** Traverse preferring edge types from the query plan or ontology ([RetrievalOntologyContext](#)), edge confidence, or recency properties — instead of uniform hop expansion.
2. **Personalized PageRank / heat-kernel proximity.** Rank expanded nodes by proximity to seed entities linked from the question (useful when Cypher returns too many symmetric neighbours).
3. **Steiner-tree / minimum subgraph for multi-entity questions.** For questions mentioning two or more anchors (“How is A related to B?”), compute a small connecting subgraph before context packing (complements [POST /graph/path](#) used in the UI).
4. **Ontology-guided traversal.** Expand along [subClassOf](#) / domain-range constraints (partially started via hierarchy expansion in [GraphExpander](#)) and align hops with OWL template ontologies.
5. **Community-aware retrieval.** [GraphExpander](#) already has a community expansion branch when [community_id](#) exists on nodes; future work: precompute communities ([ingestion/structured/graph/community_detection.py](#) uses NetworkX optionally) and use them as retrieval macros.
6. **Hybrid traversal + vector.** Combine hop expansion with HNSW entity/chunk search ([ingestion/unstructured/graph/indexes.py](#), structured hybrid stage 11) so traversal does not drift into irrelevant dense regions of the graph.
7. **Explainable traversal traces.** Extend [/retrieval/explain](#) and [ExpansionResult.path](#) with human-readable hop decisions for compliance and debugging.
8. **Benchmark-driven tuning.** Use Spider 1.0 and GraphRAG-Bench (and internal medical/semi-structured corpora) to compare hop limits, budgets, and algorithms per workspace domain.

24.4.3 Non-goals

Non-goals (unless product asks): Replacing FalkorDB with an external graph compute engine; running arbitrary user-supplied traversal code without validation.

24.5 Additional candidate scope (codebase-informed)

Items not on the slide but supported by known gaps or partial implementations:

Table 15: Codebase-informed candidate scope

Area	Candidate work
Ingestion routing	Route prose-heavy <code>.json</code> to unstructured when appropriate; optional <code>apply_ontology_for_source()</code> at ingest for filename→YAML ontology.
Knowledge alignment breadth	<code>KnowledgeAlignmentStage</code> is central to semi-structured generic branch; extend hybrid alignment to other engines where cross-document merge is needed.
Semi-structured ops	Batch reprocess and <code>ingest-pending</code> already exist; scheduled curator for stale <code>needs_review</code> queues.
Observability	Structured retrieval has metrics collector (stage 16); unify pipeline SSE + retrieval traces in one operator dashboard.
Multi-tenant isolation	<code>FalkorDBSettings.tenant_prefix</code> and per-workspace <code>graph_name</code> — document and enforce tenant boundaries for hosted deployments.
Documentation & eval artifacts	Keep <code>docs/API_REFERENCE.md</code> , <code>docs/CODEBASE.md</code> , and this generated document in sync with each release; add benchmark result appendices when Spider/GraphRAG-Bench runs exist.

24.6 How to use this section

- **Engineering managers:** Prioritize near-term items that reduce retrieval failures on customer schemas and unify the query UX.
- **Developers:** Implement against existing interfaces (`IGraphExpander`, `retrieval_service`, `PipelineOrchestrator` stages) rather than new parallel pipelines.
- **Internship / submission:** Cite this section as **future work**; distinguish it from implemented behavior in Appendix A (Codebase Reference) and Appendix B (API Reference).

When this section and the code disagree on **current** behavior, trust the code and update `docs/CODEBASE.md` in the same change.

24.7 Reference — roadmap themes (product)

Near-term themes from internal roadmap materials:

- Preserve integer PKs + fix junction-table counting (NL→Cypher on weaker schemas).
- Unify retrieval into one router (structured Cypher vs grounded prose modes).
- Complete Spider 1.0 run + expand GraphRAG-Bench beyond medical.

Longer-term themes:

- Measure layer (Observation nodes / SQL).
- Self-healing curator agent.
- Cross-pipeline ontology reconciliation + active learning from HITL review.

Graph traversing algorithms for retrieval are an explicit extension documented in §24.4 above.

25. Appendices

25.1 Glossary

Table 10: Glossary

Term	Definition
KO / Knowledge Object	Product term for SemanticObject
SemanticObject	Canonical intermediate representation (<code>models/domain.py</code>)
Workspace	Metadata record + FalkorDB <code>graph_name</code>
Engine plugin	structured unstructured template <code>semi_structured</code>
GraphRAG	Grounded retrieval with passage citations (<code>/retrieval/grounded</code>)
POLE ontology	YAML ontology labels for GLiNER unstructured extraction

25.2 Sample Cypher

```
MATCH (n) WHERE n.workspace_id = $ws RETURN labels(n), count(*)  
// Use POST /cypher with workspace-bound validation instead of ad hoc writes in  
production.
```

Figure 4: Illustrative read pattern

25.3 Related Repository Documents

- `docs/CODEBASE.md` — module map
- `docs/API_REFERENCE.md` — HTTP reference
- `README.md` — Docker and local run instructions only

Appendix A — Codebase Reference (imported)

Verbatim technical reference imported from docs`CODEBASE.md`.

BodhiECG codebase reference

This document describes **what lives where** in the repository, how data flows through the platform, and what each major module and file group is responsible for. For HTTP endpoints, see [API_REFERENCE.md](#).

1. Platform summary

BodhiECG ingests **structured** (CSV/JSON/SQL), **unstructured** (PDF/DOCX/prose), and **semi-structured** (scanned bills/invoices) sources into a **FalkorDB** knowledge graph per workspace, then answers questions via:

- **Structured retrieval** — 16-stage NL→Cypher orchestrator (`POST /query, /retrieval/nl2cypher`)
- **Grounded GraphRAG** — entity-vector grounding + passages (`POST /retrieval/grounded`)

Single execution path for ingestion:

HTTP (api/) or CLI (`app.py`)

→ `pipeline.orchestrator.PipelineOrchestrator`

→ engine plugin (structured | unstructured | template | `semi_structured`)

→ shared stages (review, ER, graph, persist, validate, index)

→ `graph.FalkorGraphClient` + storage (JSON under `data/`)

2. Top-level directory map

Path	Role
<code>api/</code>	Production FastAPI application — routers, services, schemas
<code>pipeline/</code>	Orchestrator, pipeline context, stage implementations, engine registry
<code>ingestion/</code>	Source-specific engines (structured, unstructured, <code>semi_structured</code> , template)
<code>graph/</code>	FalkorDB client, MERGE persistence, label resolution, mock fallback
<code>graph_builder/</code>	<code>SemanticObject</code> → <code>GraphNode</code> / <code>GraphEdge</code>
<code>storage/</code>	Workspace JSON, artifacts, retrieval history, entity registry
<code>models/</code>	Shared Pydantic domain types (<code>domain.py</code> , <code>canonical.py</code>)
<code>config/</code>	Settings from environment (<code>get_settings()</code>)
<code>common/</code>	Logging, exceptions, filesystem helpers, canonical stage

Path	Role
	names
<code>llm/</code>	Gateway LLM client abstraction (<code>GatewayLLMClient</code>)
<code>retrieval/</code>	Thin <code>BaseRetriever</code> interface; production logic in <code>ingestion/structured/retrieval/</code>
<code>frontend/</code>	React + TypeScript + Vite UI
<code>data/</code>	Runtime state (workspaces, uploads, caches) — bind-mounted in Docker
<code>templates/</code>	Semi-structured document templates (<code>*.json</code>)
<code>docker/</code>	Dockerfiles (<code>backend</code> , <code>frontend</code> , <code>falkordb</code>)
<code>tests/</code>	Pytest suite mirroring packages
<code>docs/</code>	Architecture and this reference
<code>notebooks/</code>	Runnable walkthroughs aligned with production modules
<code>app.py</code>	Streamlit CLI — calls same orchestrator as API

Placeholder packages (contracts / future work): `entity_resolution/`, `ontology/`, `semantic_schema/`, `normalization/`, `provenance/`, `hitl/`, `services/` at repo root — **production logic for these concerns lives inside `pipeline/` and `ingestion/` today.**

3. Production API package (`api/`)

File / folder	Responsibility
<code>main.py</code>	FastAPI app: CORS, middleware, registers all routers, <code>GET /health</code>
<code>routers/workspaces.py</code>	CRUD workspaces, stats, clear graph
<code>routers/ingestion.py</code>	Upload, detect engine, sync/async ingest, data discovery (CSV/docs/OWL/YAML), ontology generate
<code>routers/pipeline.py</code>	Run status, logs, artifacts, metrics, per-stage detail, SSE stream, resume
<code>routers/knowledge_objects.py</code>	List/get/patch KOs for a run
<code>routers/review.py</code>	Label summaries, bulk label actions, concept column mode
<code>routers/graph.py</code>	Graph snapshot, metrics, node/neighborhood, path, export, register
<code>routers/retrieval.py</code>	Query, grounded GraphRAG, NL2Cypher, plan/context/explain, stream, Cypher console, history
<code>routers/alignment.py</code>	Semantic memory + alignment decisions (semi-structured)

File / folder	Responsibility
	generic path)
<code>routers/semi_structured.py</code>	Template author/save, OCR ingest, document cache, pending ingest, OWL export
<code>services/*.py</code>	Only layer that imports <code>pipeline.*</code>, <code>graph.*</code>, <code>ingestion.*</code>, <code>storage.*</code>
<code>schemas/*.py</code>	Pydantic request/response models; envelope in <code>schemas/common.py</code>
<code>dependencies/providers.py</code>	FastAPI <code>Depends()</code> — orchestrator, workspace store, run registry
<code>events/pipeline_events.py</code>	SSE pub/sub for pipeline progress
<code>middleware/</code>	Request logging; exceptions → <code>ApiResponse</code> envelope
<code>lifecycle/lifespan.py</code>	Startup: bind asyncio loop to event bus
<code>auth/jwt_auth.py</code>	Optional JWT (disabled by default)

Rule: Routers do not contain business logic; they call services.

4. Pipeline (pipeline/)

4.1 Core modules

File	Purpose
<code>orchestrator.py</code>	Runs <code>DEFAULT_STAGES</code> in order; registers runs in <code>RunRegistry</code>
<code>context.py</code>	<code>PipelineContext</code> — mutable state for one run (documents, KOs, graph nodes/edges, artifacts, warnings)
<code>dispatcher.py</code>	<code>InputClassifier</code> — routes source → engine name
<code>registry.py</code>	<code>EngineRegistry</code> — maps <code>structured</code> , <code>unstructured</code> , <code>template</code> , <code>semi_structured</code> to engine classes
<code>execution.py</code>	<code>RunRegistry</code> , <code>RunRecord</code> — async run tracking + live context reference
<code>introspection.py</code>	Helpers to serialize context for API responses

4.2 Canonical stages (order)

Stage class	File	What it does
<code>WorkspaceStage</code>	<code>stages/workspace.py</code>	Load/create workspace, attach <code>graph_client</code>
<code>ClassificationStage</code>	<code>stages/classification.py</code>	<code>InputClassifier</code> →

Stage class	File	What it does
		<code>selected_engine</code>
IngestionStage	<code>stages/ingestion.py</code>	Invokes registered engine <code>run()</code> — parse/extract only
SemanticSchemaStage	<code>stages/semantic_schema.py</code>	LLM/schema bridge where applicable (often no-op)
KnowledgeObjectStage	<code>stages/knowledge_object.py</code>	Builds <code>SemanticObject</code> list from engine output
ReviewStage	<code>stages/review.py</code>	Auto-approve $\geq$ threshold; records review decisions
EntityResolutionStage	<code>stages/entity_resolution.py</code>	Cross-doc linker (unstructured) or <code>EntityResolutionService</code> (6 strategies) or passthrough (template)
KnowledgeAlignmentStage	<code>stages/alignment.py</code>	Merge KOs across docs — semi_structured generic branch only
GraphBuilderStage	<code>stages/graph_builder.py</code>	<code>CanonicalGraphBuilder</code> → nodes/edges (non-rejected KOs)
PersistenceStage	<code>stages/persistence.py</code>	Write to FalkorDB; chunks + MENTIONS for unstructured
ValidationStage	<code>stages/validation.py</code>	Structural + semantic validation
IndexingStage	<code>stages/indexing.py</code>	Embeddings + HNSW/BM25 — unstructured engine only
RetrievalReadyStage	<code>stages/retrieval.py</code>	Marks retrieval metadata; caches orchestrator hook

Supporting: `stages/cross_doc_link.py` — plans cross-document links using chunk embeddings.

Alignment implementation: `alignment/knowledge_alignment.py` — hybrid scorer (exact, fuzzy, ontology, embedding, LLM).

4.3 Engine plugins (pipeline/engines/)

Engine	File	Ingestion responsibility
structured	<code>structured_engine.py</code>	Parsers + chunker + spaCy/GLiNER NER → entities and relations
unstructured	<code>unstructured_engine.py</code>	Delegates to <code>ingestion.unstructured.knowledge_object.pipeline</code>
template	<code>template_engine.py</code>	Delegates to structured LangGraph CSV+OWL pipeline
semi_structured	<code>semi_structured_engine.py</code>	Delegates to semi-structured

Engine	File	Ingestion responsibility
		LangGraph (template vs generic branch)

4.4 Classification rules (`dispatcher.py`)

- **template** — explicit `template_id`, or OWL ontology + tabular extension
- **structured** — `.csv`, `.xlsx`, `.xls`, `.json`, `.sql` without ontology routing to template
- **semi_structured** — `.pdf` / images if `templates/*.json` exists
- **unstructured** — other document extensions or content sniff → prose

Override: `engine_hint` on ingest request.

5. Ingestion engines (`ingestion/`)

5.1 Structured (`ingestion/structured/`)

Large legacy + production tree. Key areas:

Area	Path	Notes
Parsers	<code>parsers/</code>	CSV, JSON, SQL, XLSX, PDF, image OCR cascade
Chunking	<code>chunking/</code>	<code>ChunkerOrchestrator</code> , hierarchical/recursive
NER	<code>ner/</code>	<code>HybridNEREngine</code> , spaCy + optional GLiNER
Template KO	<code>knowledge_object/</code>	Builder, ontology parser, schema builder
LangGraph ingest	<code>orchestration/</code> <code>langgraph_pipeline.py</code>	Template engine internal flow
Entity resolution	<code>entity_resolution/</code>	6-strategy <code>EntityResolutionService</code> , RapidFuzz
Retrieval	<code>retrieval/orchestration/</code> <code>orchestrator.py</code>	16-stage production retrieval
Legacy API	<code>api/v1/*.py</code>	Not mounted on <code>api.main</code>

5.2 Unstructured (`ingestion/unstructured/`)

Area	Path	Notes
KO pipeline	<code>knowledge_object/</code> <code>pipeline.py</code>	parse → chunk → extract → resolve
Parse	<code>cocoindex_flow/</code> <code>parse_fn.py</code>	Docling (+ optional vision)

Area	Path	Notes
Extract	<code>extraction/relex.py</code>	GLiNER-Relex joint NER+relations
Clean	<code>extraction/clean.py</code>	Boilerplate filter, span de-fragmentation
Coref	<code>extraction/coref.py</code>	fastcoref (optional)
Resolution	<code>resolution/merge.py</code> , <code>scoring.py</code> , <code>linker.py</code>	Within-doc + cross-doc identity
Ontology	<code>ontology/loader.py</code> , <code>config/*.yaml</code> , <code>resolver.py</code>	Labels for GLiNER; auto-suggest by filename (resolver exists, wire at ingest optional)
Graph	<code>graph/context.py</code>	Grounded GraphRAG assembly
Graph	<code>graph/indexes.py</code> , <code>connection.py</code>	HNSW, BM25, workspace graph names
Graph	<code>graph/retrieval_substrate.py</code>	Backfill embeddings (module present; wire to API optional)
LLM	<code>llm/gateway.py</code> , <code>healthcheck.py</code>	Embeddings, chat, vision via gateway
Legacy API	<code>api/routes/</code>	Direct ingest/query — bypass unified pipeline

5.3 Semi-structured (`ingestion/semi_structured/`)

File / area	Purpose
<code>ocr.py</code>	Tesseract / pypdf / Docling / EasyOCR fallback
<code>document_templates.py</code>	Load <code>templates/*.json</code> , classify by signature
<code>template_extractor.py</code>	Deterministic regex extraction
<code>template_ko_builder.py</code>	KOs + <code>EntityRegistryStore</code> business keys
<code>kv_extraction.py</code> , <code>business_schema.py</code>	Generic branch KV + LLM schema
<code>orchestration/langgraph_pipeline.py</code>	Template vs generic graph
<code>document_cache.py</code>	<code>pending</code> / <code>needs_review</code> / <code>ingested</code> JSON cache

5.4 Template package (`ingestion/template/`)

Shared deterministic template execution helpers (see package README).

6. Graph layer (`graph/`)

Module	Purpose
<code>falkordb.py</code>	<code>FalkorGraphClient</code> — MERGE nodes/edges,

Module	Purpose
	schema indexes, mock fallback
<code>base.py</code>	<code>BaseGraphClient</code> interface
<code>mock.py</code>	In-memory graph when DB unavailable
<code>label_resolver.py</code>	Map relation types → FalkorDB edge labels
<code>cypher_normalize.py</code>	Normalize query results for JSON/API

7. Graph builder (`graph_builder/`)

Module	Purpose
<code>canonical.py</code>	<code>CanonicalGraphBuilder</code> — <code>SemanticObject</code> → graph nodes and typed edges

8. Storage (`storage/`)

Module	Purpose
<code>workspace_store.py</code>	<code>data/workspaces/workspaces.json</code> — workspace metadata + graph name
<code>entity_registry_store.py</code>	Per-workspace business-key registry (semi-structured OCR dedupe)
<code>retrieval_history_store.py</code>	Query history JSON
<code>artifact_store.py</code>	Pipeline artifact paths
Under <code>data/workspaces/</code>	<code>knowledge_objects/</code> , <code>semantic_memory/</code> , <code>alignment_decisions/</code> , etc.

9. Models (`models/`)

File	Purpose
<code>domain.py</code>	<code>SemanticObject</code> , <code>Entity</code> , pipeline stage enums, confidence, provenance
<code>canonical.py</code>	Document/chunk canonical shapes shared across engines

10. Config & environment (`config/`)

Source	Purpose
<code>settings.py</code>	Pydantic-settings: FalkorDB, API, LLM gateway, storage paths
<code>.env</code>	Secrets and overrides (not committed)

Source	Purpose
<code>.env.example</code>	Template for all important variables

Common variables: `FALKORDB_HOST`, `LLM_GATEWAY_*`, `LLM_GATEWAY_EMBED_MODEL`, `EMBED_DIM`, `ONTOLOGY_PATH`, `EMBED_ON_INGEST`, `TESSERACT_CMD`, `OCR_FALLBACK`.

11. LLM (`llm/` vs `ingestion/unstructured/llm/`)

Package	Purpose
<code>llm/client.py</code>	Generic gateway HTTP client
<code>ingestion/unstructured/llm/gateway.py</code>	Embeddings, chat, vision used by pipeline + retrieval
<code>ingestion/unstructured/llm/healthcheck.py</code>	Connectivity test for chat/embed/vision

Structured retrieval and Cypher generation use gateway clients under `ingestion/structured/`.

12. Retrieval (where code lives)

Mode	Primary code
Structured 16-stage	<code>ingestion/structured/retrieval/orchestration/orchestrator.py</code>
API wiring	<code>api/services/retrieval_service.py</code>
Grounded GraphRAG	<code>ingestion/unstructured/graph/context.py</code>
FalkorDB vector/BM25	<code>ingestion/structured/retrieval/vector/falkordb_search.py</code>
Entity linking (retrieval)	<code>ingestion/structured/retrieval/entity_linking/</code>

13. Frontend (`frontend/`)

Area	Purpose
<code>src/pages/</code>	Ingestion, Pipeline, Retrieval, Graph, Workspaces, etc.
<code>src/api/</code>	Typed fetch wrappers (<code>client.ts</code> , <code>retrieval.ts</code> , ...)
<code>src/hooks/</code>	React Query / pipeline stream hooks
<code>src/features/</code>	Ingestion wizards, semi-structured dropzones, dev panel
<code>vite.config.ts</code>	Dev proxy <code>/api</code> → backend <code>:8000</code>
Docker nginx	<code>docker/frontend/nginx.conf</code> — same <code>/api</code> proxy in production

14. Docker

File	Purpose
<code>docker-compose.yml</code>	FalkorDB + backend + frontend
<code>docker/backend/Dockerfile</code>	Multi-stage Python 3.12, uv, OCR deps
<code>docker/backend/entrypoint.sh</code>	Creates <code>data/*</code> dirs at container start
<code>docker-compose.registry.yml</code>	Pull pre-built images from Docker Hub
<code>.dockerignore</code>	Excludes <code>data/</code> , <code>frontend/</code> , tests from backend image context

15. Tests (tests/)

Folder	Coverage
<code>tests/api/</code>	HTTP contract tests
<code>tests/pipeline/</code>	Classifier, graph builder, semi-structured flows
<code>tests/structured/</code>	NL→Cypher, retrieval, template pipeline
<code>tests/unstructured/</code>	Models, retrieval substrate helpers
<code>tests/semi_structured/</code>	OCR, template extraction

Run: `uv run pytest` (see `tests/README.md`).

16. Legacy and parallel paths (avoid in new integrations)

Path	Issue
<code>ingestion/structured/app.py</code>	Standalone terminal pipeline
<code>ingestion/structured/api/main.py</code>	Duplicate REST surface
<code>ingestion/unstructured/orchestration/pipeline.py</code>	Writes graph directly, skips unified stages
<code>ingestion/unstructured/cocoindex_flow/flow.py</code>	Incremental CocolIndex batch
<code>app.py</code>	Streamlit — fine for demos, not the production UI

17. Key design decisions (quick answers for reviews)

Question	Answer
One graph per workspace?	Yes — <code>Workspace.graph_name</code> in FalkorDB
Where are KOs before graph?	<code>PipelineContext.knowledge_objects</code> → <code>resolved_objects</code>
Deduplication	Within-doc: unstructured <code>merge.py</code> ; cross-doc: linker or

Question	Answer
	ER service; template: business keys; graph: MERGE on ids
Relations from CSV?	Yes — both the template engine and the generic structured NER path
Vector search	Unstructured ingest indexing; grounded retrieval; optional in structured retrieval stage 11
Human review	Review stage + semi-structured <code>needs_review</code> cache

18. Related documents

- `API_REFERENCE.md` — HTTP API
- `UnifiedPipelineArchitecture.md`
- `StructuredKnowledgeObjectImplementation.md`
- `UnstructuredKnowledgeObjectImplementation.md`
- `FrontendArchitecture.md`

When this document and code disagree, **trust the code** and update this file in the same PR.

Appendix B — API Reference (imported)

Verbatim technical reference imported from docs`API\_REFERENCE.md`.

BodhiECG API reference

Production HTTP API served by `api.main:app` (`uvicorn api.main:app`).

Resource	URL
OpenAPI (Swagger)	<code>http://localhost:8000/docs</code>
ReDoc	<code>http://localhost:8000/redoc</code>
Health	<code>GET /health</code>

How to call the API

How you run the app	Base URL for routes below
Backend directly	<code>http://localhost:8000</code> + path (e.g. <code>/workspaces</code>)
Docker UI (nginx)	<code>http://localhost:8080/api</code> + path (e.g. <code>/api/workspaces</code>)
Local Vite dev	Same as Docker UI — Vite proxies <code>/api/*</code> → backend with the <code>/api</code> prefix stripped

All JSON endpoints (except SSE streams) return the same envelope described below. Interactive schemas match the Pydantic models in `api/schemas/`.

Response envelope

Every successful JSON handler returns:

```
{
  "status": "success",
  "message": "optional human-readable summary",
  "data": { },
  "errors": [],
  "warnings": []
}
```

On failure, FastAPI typically returns `{"detail": ...}` with an HTTP status (400, 404, 422, etc.). Pipeline completion with stage errors may still return `status: \"success\"` with warnings/errors inside `data` (see `POST /ingest`).

Endpoint index

Method	Path	Tag
GET	<code>/health</code>	system

Method	Path	Tag
GET	/workspaces	workspaces
POST	/workspaces	workspaces
GET	/workspaces/{workspace_id}	workspaces
PATCH	/workspaces/{workspace_id}	workspaces
DELETE	/workspaces/{workspace_id}	workspaces
GET	/workspaces/{workspace_id}/stats	workspaces
POST	/workspaces/{workspace_id}/clear-graph	workspaces
GET	/data/csv	ingestion
GET	/data/documents	ingestion
GET	/data/ontologies	ingestion
GET	/data/ontologies/preview	ingestion
GET	/data/ontologies/unstructured	ingestion
POST	/data/ontologies/unstructured/generate	ingestion
POST	/upload	ingestion
POST	/ingest/detect	ingestion
POST	/ingest	ingestion
POST	/pipeline/start	ingestion
GET	/pipeline/{run_id}	pipeline
GET	/pipeline/{run_id}/status	pipeline
GET	/pipeline/{run_id}/logs	pipeline
GET	/pipeline/{run_id}/artifacts	pipeline
GET	/pipeline/{run_id}/metrics	pipeline
GET	/pipeline/{run_id}/stage/{stage}	pipeline

Method	Path	Tag
POST	/pipeline/{run_id}/resume	pipeline
GET	/pipeline/{run_id}/stream	pipeline (SSE)
GET	/knowledge-objects	knowledge-objects
GET	/knowledge-objects/{canonical_id}	knowledge-objects
PATCH	/knowledge-objects/{canonical_id}	knowledge-objects
GET	/review/concept-columns	review
POST	/review/concept-mode	review
GET	/review/labels	review
POST	/review/label-action	review
GET	/graph/list	graph
POST	/graph/register	graph
GET	/graph	graph
GET	/graph/metrics	graph
GET	/graph/node/{node_id}	graph
GET	/graph/neighborhood/{node_id}	graph
POST	/graph/path	graph
POST	/graph/export	graph
POST	/query	retrieval
POST	/retrieval/grounded	retrieval
POST	/retrieval/nl2cypher	retrieval
POST	/retrieval/plan	retrieval
POST	/retrieval/context	retrieval
POST	/retrieval/explain	retrieval
POST	/retrieval/query/stream	retrieval (SSE)
GET	/retrieval/history	retrieval
POST	/cypher/validate	retrieval
POST	/cypher	retrieval

Method	Path	Tag
GET	/alignment/{workspace_id}/semantic-memory	alignment
GET	/alignment/{workspace_id}/decisions	alignment
POST	/alignment/override	alignment
GET	/semi-structured/templates	semi-structured
GET	/semi-structured/templates/{domain}	semi-structured
POST	/semi-structured/templates/author	semi-structured
POST	/semi-structured/templates/draft	semi-structured
POST	/semi-structured/templates/save	semi-structured
POST	/semi-structured/ingest	semi-structured
POST	/semi-structured/ingest/path	semi-structured
GET	/semi-structured/documents	semi-structured
GET	/semi-structured/status	semi-structured
GET	/semi-structured/workspace/{workspace_id}/entities	semi-structured
POST	/semi-structured/documents/{doc_id}/resolve	semi-structured
POST	/semi-structured/documents/{doc_id}/mark-ingested	semi-structured
POST	/semi-structured/documents/reprocess	semi-structured
POST	/semi-structured/documents/ingest-pending	semi-structured
POST	/semi-structured/documents/{doc_id}/reprocess	semi-structured
POST	/semi-structured/	semi-structured

Method	Path	Tag
	ontology/export	
POST	/semi-structured/export/ csv	semi-structured

System

GET /health

Liveness check.

Response data: `{"status": "ok"}`

Workspaces

Workspace metadata and FalkorDB graph binding live in `storage.workspace_store` (JSON under `data/workspaces/`).

GET /workspaces

List all workspaces.

Response data: array of `WorkspaceResponse` — `workspace_id`, `workspace_name`, `graph_name`, `description`, `status`, `document_count`, `ontology_used`, `created_at`.

POST /workspaces

Create a workspace.

Body: `WorkspaceCreateRequest`

Field	Type	Required
name	string	yes
description	string	no (default <code>"\"</code>)

Status: 201 on success. **Errors:** 400 if name invalid or duplicate.

GET /workspaces/{workspace_id}

Errors: 404 if unknown workspace.

PATCH /workspaces/{workspace_id}

Rename workspace.

Body: `WorkspaceRenameRequest` — `{ "name": "..."`

Errors: 400 validation, 404 not found.

DELETE /workspaces/{workspace_id}

Delete workspace record (does not necessarily drop the FalkorDB graph unless implemented in service).

Errors: 404 not found.

GET /workspaces/{workspace_id}/stats

Aggregated counts and active pipeline run pointer.

Response data: `WorkspaceStatsResponse` — `dataset_count`, `ontology_count`, `node_count`, `edge_count`, `active_pipeline_run_id`, `active_pipeline_status`, etc.

POST /workspaces/{workspace_id}/clear-graph

Delete all nodes and edges in the workspace's graph; keeps the workspace. Use before re-ingesting after graph logic fixes.

Response data: service-specific summary dict.

Ingestion & data staging

Routes in `api/routers/ingestion.py` (no path prefix). Files are staged on disk; pipeline execution goes through `PipelineOrchestrator`.

GET /data/csv

List CSV files available under the project data area for structured/template flows.

Response data: `UploadedFileResponse[]` — `name`, `path`, `size_bytes`, `content_type`.

GET /data/documents

List document files (PDF, DOCX, etc.) for unstructured ingestion.

GET /data/ontologies

List OWL ontologies for the **template** engine (CSV + ontology graph build).

GET /data/ontologies/preview?path={path}

Parse OWL and return classes, relationships, subclasses, and stats for UI preview.

Query: `path` — server-side path returned from list endpoints.

Errors: 404 not found, 422 invalid ontology.

GET /data/ontologies/unstructured

List YAML POLE-style ontologies for structured(NER) / unstructured GLiNER extraction.

POST /data/ontologies/unstructured/generate

LLM-generate a YAML ontology from document text when none is selected.

Body: `GenerateOntologyRequest`

Field	Type	Notes
<code>source</code>	string	Document path to sample
<code>domain_hint</code>	string	optional
<code>sample_chunks</code>	int	default 40

Response data: `GenerateOntologyResponse` — `path`, `entity_types`, `relation_types`, etc.

POST /upload?workspace_id={id}

Multipart upload; stages file only (no pipeline).

Form: `file` (required)

Response data: `UploadedFileResponse` with `path` to pass into `POST /pipeline/start` or `POST /ingest`.

`POST /ingest/detect`

Preview which ingestion **engine** would run (same rules as `ClassificationStage`).

Body: `DetectEngineRequest`

Field	Type
<code>source</code>	string
<code>template_id</code>	string null
<code>ontology_id</code>	string null
<code>ontology_path</code>	string null
<code>source_type</code>	string null

Response data: `DetectEngineResponse` — `engine`, `reason`, `signals`.

`POST /ingest`

Synchronous full pipeline — blocks until complete. Prefer async flow for UI progress.

Body: `IngestRequest`

Field	Type	Notes
<code>source</code>	string	Primary file path
<code>workspace_id / workspace_name</code>	string	optional
<code>engine_hint</code>	string	<code>structured</code> <code>unstructured</code> <code>template</code> ...
<code>template_id</code>	string	template flow
<code>ontology_id / ontology_path</code>	string	template OWL
<code>csv_file_paths</code>	string[]	multi-CSV template ingest
<code>source_type</code>	string	optional classifier hint
<code>ontology_path_unstructured</code>	string	YAML for GLiNER
<code>auto_generate_ontology</code>	bool	LLM ontology if missing

Response data: `IngestResponse` — `run_id`, `counts`, `warnings`, `errors`, `execution_timings`.

`POST /pipeline/start`

Async pipeline start (same body as `IngestRequest`).

Status: 202 Accepted

Response data: PipelineStatusResponse — run_id, status: \"running\". Poll GET /pipeline/{run_id}/status or subscribe to GET /pipeline/{run_id}/stream.

Pipeline runs

Prefix: /pipeline. Run records are in-memory for the process lifetime (see pipeline.execution.RunRegistry).

GET /pipeline/{run_id}

Full run summary: run_id, status, selected_engine, workspace_id, summary.

GET /pipeline/{run_id}/status

Lightweight status + summary.

GET /pipeline/{run_id}/logs

Response data: { \"run_id\", \"log_lines\": string[] }

GET /pipeline/{run_id}/artifacts

Intermediate artifacts: dataframes, joins, schema, ontology parse, classification counts, semi-structured identity_report, etc.

GET /pipeline/{run_id}/metrics

Timing, KO/graph counts, optional validation metrics.

GET /pipeline/{run_id}/stage/{stage}

Per-stage detail (name matches orchestrator stage id).

POST /pipeline/{run_id}/resume

Resume a **paused** run. **Errors:** 400 if not paused.

Response data: { \"status\": \"resumed\", \"run_id\" }

GET /pipeline/{run_id}/stream

Server-Sent Events (text/event-stream). Replays completed stages, then live events until pipeline_complete, pipeline_failed, or pipeline_paused. Heartbeat comments every 15s when idle.

Headers: Cache-Control: no-cache, X-Accel-Buffering: no (nginx-friendly).

Knowledge objects

Prefix: /knowledge-objects. Scoped to a pipeline **run** (in-memory KOs for that run).

GET /knowledge-objects?run_id={run_id}&...

List/filter KOs.

Query	Type	Default
run_id	string	required
entity_type	string	
validation_status	string	

Query	Type	Default
<code>min_confidence</code>	float	
<code>source_table</code>	string	
<code>search</code>	string	
<code>limit</code>	int	200 (max 2000)
<code>offset</code>	int	0

Response data: `KnowledgeObjectListResponse` — `total`, `filtered`, `items[]`.

GET `/knowledge-objects/{canonical_id}?run_id={run_id}`

Single KO with properties and relationships.

PATCH `/knowledge-objects/{canonical_id}?run_id={run_id}`

Human edit.

Body: `KnowledgeObjectPatchRequest` — optional `validation_status`, `display_name`, `properties` (dict), `reason`, `actor`.

Human review (structured pipeline)

Prefix: `/review`. Uses `run_id` query param or in JSON body depending on route.

GET `/review/concept-columns?run_id={run_id}`

Columns classified as **Concept** with current concept vs property mode.

POST `/review/concept-mode`

Body: `ConceptModeRequest` — `run_id`, `source_table`, `column`, `mode` (`concept` | `property`), `actor`, `reason`.

GET `/review/labels?run_id={run_id}`

Per-entity-type counts: `approved` / `needs_review` / `rejected`.

POST `/review/label-action`

Bulk approve, reject, or remove an entire label.

Body: `LabelActionRequest` — `run_id`, `entity_type`, `action` (`approve` | `reject` | `remove`), `actor`, `reason`.

Graph explorer

Prefix: `/graph`. Reads FalkorDB via workspace → `graph_name`.

GET `/graph/list`

All graphs on the server with counts and owning workspace (if registered).

POST `/graph/register`

Adopt an existing FalkorDB graph as a workspace.

Body: `RegisterGraphRequest` — `graph_name`, optional `name`.

Status: 201. **Errors:** 400 on workspace errors.

GET /graph?workspace_id={id}&limit={n}

Full snapshot for explorer (default `limit=2000`). Avoid polling on every UI interaction.

GET /graph/metrics?workspace_id={id}

Node/edge counts, label and relationship type histograms, density.

GET /graph/node/{node_id}?workspace_id={id}

Single node properties and labels.

GET /graph/neighborhood/{node_id}?workspace_id={id}&hops={n}&limit={n}

Ego network snapshot (default `hops=1`, `limit=200`).

POST /graph/path

Body: `PathRequest` — `workspace_id`, `source_id`, `target_id`, `max_hops` (default 4).

POST /graph/export

Body: `GraphExportRequest` — `workspace_id`, `format` (`json` | `cypher`).

Retrieval & Cypher

Routes in `api/routers/retrieval.py` (mixed prefixes). Requires LLM gateway configuration for NL and grounded paths.

POST /query

Primary **structured retrieval** — 16-stage NL→Cypher orchestrator.

Body: `QueryRequest` — `workspace_id`, `query`, `top_k` (default 20).

Response data: `QueryResponse` — `answer`, `cypher_used`, citations, graph subgraph, `pipeline_stages`, latency, grounding metrics.

POST /retrieval/grounded

GraphRAG — entity-vector grounding, typed subgraph, source passages, cited answer. Same response shape as `/query`.

Body: `GroundedQueryRequest` — adds optional `rewrite` (default true) for query expansion.

POST /retrieval/nl2cypher

Generate (and optionally execute) Cypher from natural language.

Body: `NL2CypherRequest` — `workspace_id`, `query`, `max_hops`, `execute` (default true).

Response data: `NL2CypherResponse` — plan, `generated_cypher`, validation, rows/columns if executed, `self_correction_attempts`.

POST /retrieval/plan

Intent/entity/plan preview without full answer generation.

Body: `RetrievalPlanRequest`

POST /retrieval/context

Retrieve ranked nodes, relationships, chunks, and context sections for a query (no final answer synthesis in the same shape as full query).

Body: `QueryRequest`

POST /retrieval/explain

Explainability bundle: entities, traversal, chunks, reasoning path, final answer.

Body: `RetrievalExplainRequest` — `workspace_id` required (400 if missing).

POST /retrieval/query/stream

SSE stream of retrieval progress (same body as `QueryRequest`).

GET /retrieval/history?workspace_id={id}&limit={n}

Recent queries (default `limit=50`, optional workspace filter).

POST /cypher/validate

Syntax/security validation only.

Body: `CypherValidateRequest` — `cypher` (and `workspace_id` in schema; validation uses `cypher` only).

Response data: { `"valid"`: bool, `"message"`: string }

POST /cypher

Execute validated read Cypher against the workspace graph.

Body: `CypherRequest` — `workspace_id`, `cypher`, `params` (default {}).

Errors: 400 if validation fails.

Response data: `CypherResponse` — `columns`, `rows`, `row_count`, `execution_ms`.

Knowledge alignment (semi-structured / generic branch)

Prefix: `/alignment`. Decisions persisted per workspace under `data/`.

GET /alignment/{workspace_id}/semantic-memory

Compact vocabulary the semi-structured LLM schema step reuses (entity types, relationship types, frequent concepts).

Errors: 404 if workspace unknown.

GET /alignment/{workspace_id}/decisions

Last alignment pass: each new KO, candidates, scores, merge/create/review outcome.

POST /alignment/override

Human override for `needs_review` decisions.

Body: `AlignmentOverrideRequest`

Field	Notes
<code>decision</code>	<code>merge</code> <code>create_new</code> <code>needs_review</code>
<code>target_ko_id</code>	required when <code>decision</code> is <code>merge</code>

Field	Notes
-------	-------

Errors: 404, 400 validation.

Semi-structured (template OCR pipeline)

Prefix: `/semi-structured`. Templates live in `templates/*.json`; document cache in `data/semi_structured/cache/`.

Typical flow: author template → `POST /semi-structured/ingest` → review queue → `ingest-pending` or auto pipeline → graph.

Templates

Method	Path	Purpose
GET	<code>/semi-structured/templates</code>	List template summaries
GET	<code>/semi-structured/templates/{domain}</code>	Full JSON for one domain
POST	<code>/semi-structured/templates/author</code>	LLM author from sample paths (AuthorTemplateRequest)
POST	<code>/semi-structured/templates/draft</code>	Multipart: <code>file</code> , optional <code>domain</code> , <code>source_subdir</code> — draft without saving
POST	<code>/semi-structured/templates/save</code>	Persist reviewed template (SaveTemplateRequest: <code>template</code> , <code>save_mode</code> <code>new</code> <code>replace</code> , <code>replace_domain</code>)

Ingest scans

POST `/semi-structured/ingest`

Multipart: `file` (required).

Query / form	Default	Notes
<code>auto_pipeline</code>	<code>true</code>	Runs full orchestrator after cache staging
<code>workspace_id / workspace_name</code>	<code>null</code>	Target workspace for graph pipeline
<code>domain</code>	<code>null</code>	Skip signature match; force template
<code>template_json</code>	<code>null</code>	Form field: inline template override (JSON object)

Response data: `IngestDocumentResponse` — `document_id`, `status` (`pending` | `needs_review` | ...), `pipeline_run_id` if auto pipeline.

POST /semi-structured/ingest/path

Same as ingest for a path already on disk. Query params: `path`, `auto_pipeline`, `workspace_id`, `workspace_name`, `domain`, optional `template_json` string.

Document cache & workspace entities

Method	Path	Purpose
GET	/semi-structured/documents?status=	Filter: pending, needs_review, ingested
GET	/semi-structured/status	Bucket counts
GET	/semi-structured/workspace/{workspace_id}/entities	Cross-document merged entities (MRN, provider keys, etc.)
POST	/semi-structured/documents/{doc_id}/resolve	Move <code>needs_review</code> → <code>pending</code>
POST	/semi-structured/documents/{doc_id}/mark-ingested	Mark consumed by downstream pipeline
POST	/semi-structured/documents/reprocess?status=needs_review	Batch re-OCR/re-extract
POST	/semi-structured/documents/{doc_id}/reprocess	Single-document reprocess
POST	/semi-structured/documents/ingest-pending	Queue graph pipeline for all pending (<code>limit</code> , <code>workspace_id</code> , <code>workspace_name</code>)

Exports

Method	Path	Purpose
POST	/semi-structured/ontology/export	Write per-domain + combined OWL from templates
POST	/semi-structured/export/csv	Consolidated CSV of cached records

Environment dependencies

Feature	Typical env vars
FalkorDB	FALKORDB_HOST, FALKORDB_PORT
LLM gateway	LLM_GATEWAY_BASE_URL, LLM_GATEWAY_JWT

Feature	Typical env vars
Unstructured NER	ONTOLOGY_PATH, Hugging Face cache
OCR (local dev)	TESSERACT_CMD

See `.env.example` and root `README.md` for run/setup (Docker vs local).

Related docs

- `CODEBASE.md` — module layout and data flow
- OpenAPI at `/docs` — always authoritative for request field renames

Appendix C — Run and Deployment Guide (from README.md)

Verbatim technical reference imported from [README.md](#).

BodhiECG

Unified enterprise Knowledge Graph platform — ingest structured CSVs, unstructured documents, and semi-structured scans (invoices, medical bills) into a FalkorDB graph, then query with natural language (NL→Cypher) or GraphRAG.

There are **two ways to run** BodhiECG:

Mode	Best for	UI URL	How to start
Docker Compose	Deployment, demos, sharing with others	http://localhost:8080	<code>docker compose up -d</code>
Local dev	Active development (hot reload)	http://localhost:5173	<code>uvicorn + npm run dev</code>

Use **one mode at a time** — both bind backend port **8000** and will conflict if run together.

Production API endpoint: `uvicorn api.main:app` — not the legacy `ingestion/structured/api` or `ingestion/unstructured/api` apps.

Architecture at a glance

Browser



Data flow: Frontend → Backend API → `PipelineOrchestrator` → engines (structured / unstructured / semi-structured / template) → FalkorDB + JSON workspace stores under `data/`.

Layer	Technology	Docker port	Local dev port
Graph DB	FalkorDB	6379	6379
FalkorDB Browser	Web UI in FalkorDB image	http://localhost:3000	http://localhost:3000
Backend API	FastAPI + Uvicorn	http://localhost:8000	http://localhost:8000
Frontend	React + Vite / nginx	http://localhost:8080	http://localhost:5173

Prerequisites

Docker Compose (run full stack)

Requirement	Notes
Docker Desktop	With Compose v2 (<code>docker compose</code>)
<code>.env</code> file	Copy from <code>.env.example</code> ; set <code>LLM_GATEWAY_JWT</code> for retrieval/LLM features

No local Python or Node install required when using Docker.

Local development (code changes)

Requirement	Version / notes
Python	3.12+
uv	install
Node.js	20+
Docker	For FalkorDB (or use compose for DB only)
Tesseract OCR	Image scans only (<code>.jpg</code> , <code>.png</code>); PDFs use Docling
LLM Gateway	Required for NL→Cypher, GraphRAG, template authoring

Windows Tesseract: install from [UB-Mannheim](#), then set `TESSERACT_CMD=C:\Program Files\Tesseract-OCR\tesseract.exe` in `.env`.

Quick start — Docker Compose (recommended)

Runs FalkorDB + backend + frontend in three containers. When this is up, open <http://localhost:8080> — that is the full app.

1. Configure environment

```
cp .env.example .env
```

Edit `.env` — at minimum set `LLM_GATEWAY_JWT` and `LLM_GATEWAY_BASE_URL` if you use retrieval or LLM features.

Compose overrides these for the backend container automatically:

- `FALKORDB_HOST=falkordb` (Docker service name, not `localhost`)
- `TESSERACT_CMD=/usr/bin/tesseract`

2. Build and start

```
docker compose build # first backend build ~15–20 min
```

```
docker compose up -d
```

3. Verify

```
docker compose ps
```

All three services should show `Up (healthy)`.

```
# Linux / macOS / Git Bash
curl http://localhost:8000/health
curl http://localhost:8080/api/health
```

```
# Windows PowerShell — use curl.exe (curl is an alias for Invoke-WebRequest)
curl.exe http://localhost:8000/health
curl.exe http://localhost:8080/api/health
```

Expected: `{\"status\": \"ok\"}` for both.

Open the UI: `http://localhost:8080`

API docs (direct): `http://localhost:8000/docs`

FalkorDB browser: `http://localhost:3000`

4. Stop / restart

```
docker compose logs -f backend # tail logs
docker compose down # stop containers
docker compose down -v # stop + delete FalkorDB graph volume
docker compose up -d # start again
```

Docker file layout

```
docker/
├── falkordb/Dockerfile # FROM falkordb/falkordb (pinned wrapper)
├── backend/
│   ├── Dockerfile # Python 3.12 + uv + tesseract
│   └── entrypoint.sh # creates data/ dirs on startup
├── frontend/
│   ├── Dockerfile # node build → nginx
│   └── nginx.conf # static files + /api proxy (SSE-safe)
├── docker-compose.yml
└── .dockerignore
```

The nginx config proxies `/api/*` → backend (same pattern as Vite dev), so no `VITE_API_BASE_URL` build arg is needed.

Data persistence (Docker)

Path	Purpose
<code>./data</code> (bind mount)	Workspaces, uploads, semi-structured cache
<code>./templates</code> (bind mount, read-only)	Document templates — edits apply without rebuilding backend
<code>falkordb_data</code> (named volume)	Graph database files

Docker and local dev share the same `./data` directory, so cache counts (`pending`, `needs_review`, `ingested`) stay in sync.

Keeping Docker in sync with the codebase

What changed	Action
Python backend (<code>api/</code> , <code>ingestion/</code> , <code>pipeline/</code> , ...)	Rebuild backend image
React frontend (<code>frontend/src/</code> , ...)	Rebuild frontend image
Template JSON (<code>templates/*.json</code>)	No rebuild — live via bind mount
Runtime data (<code>data/</code>)	No rebuild — live via bind mount

After pulling or editing backend/frontend code:

```
docker compose build backend frontend
```

```
docker compose up -d --force-recreate backend frontend
```

Verify the running backend has the latest routes:

```
curl.exe http://localhost:8080/api/semi-structured/status
```

```
curl.exe -X POST http://localhost:8080/api/semi-structured/documents/reprocess?status=needs_review
```

Should return JSON (200), not 404

Semi-structured review queue: if documents are stuck in `needs_review` after template fixes, reprocess them (UI: *Reprocess review queue*, or API above), then run graph ingestion for pending docs (*Ingest pending* button or `POST /semi-structured/documents/ingest-pending`).

Quick start — Local dev

Use this when editing Python or React code with hot reload.

1. Clone and configure

```
cd BodhiECG
cp .env.example .env
```

2. Start FalkorDB

Either use compose for DB only:

```
docker compose up -d falkordb
```

Or a standalone container:

```
docker run -d --name bodhi-falkordb \ -p 6379:6379 -p 3000:3000 \ falkordb/falkordb:latest
```

Verify: `redis-cli -h localhost -p 6379 ping` → PONG

Port conflict: if you already ran `docker compose up -d` (full stack), FalkorDB is already on 6379/3000 — skip this step.

3. Start the backend

```
uv sync
```

```
uv run uvicorn api.main:app --reload --host 0.0.0.0 --port 8000
```

Verify: `http://localhost:8000/health`

4. Start the frontend

```
cd frontend
npm install
npm run dev
```

Open: <http://localhost:5173>

Vite proxies </api/> → <http://localhost:8000> (see [frontend/vite.config.ts](#)).

Docker vs local dev

	Docker Compose	Local dev
Start	<code>docker compose up -d</code>	<code>uvicorn + npm run dev</code>
Frontend URL	http://localhost:8080	http://localhost:5173
Backend URL	http://localhost:8000 or /api/ via :8080	http://localhost:8000
Hot reload	No — rebuild backend/frontend images after code changes (templates + data/ are bind-mounted live)	Yes
Tesseract	Built into backend image	Install on host
Share with others	Push images to a registry (see below)	Share repo + build instructions

When Docker is running, you do **not** need separate `uvicorn` or `npm run dev` processes — the containers are the frontend and backend.

Port reference

Service	Docker	Local dev	Purpose
Frontend UI	http://localhost:8080	http://localhost:5173	React app
Backend API	http://localhost:8000	http://localhost:8000	REST + SSE
API via UI proxy	http://localhost:8080/api/...	http://localhost:5173/api/...	Same-origin API calls
API health	/health or /api/health	/health	Liveness
OpenAPI	http://localhost:8000/docs	http://localhost:8000/docs	Swagger UI
FalkorDB (Redis)	localhost:6379	localhost:6379	Graph storage
FalkorDB Browser	http://localhost:3000	http://localhost:3000	Graph explorer

Override FalkorDB host ports if 6379/3000 are taken:

```
# Windows
set FALKORDB_REDIS_PORT=6380
```

```
set FALKORDB_BROWSER_PORT=3001
docker compose up -d
```

Backend still connects internally via `falkordb:6379` on the compose network.

Sharing Docker images with others

Option A — Share the repo (simplest)

Recipients clone, configure `.env`, and build:

```
git clone
cd BodhiECG
cp .env.example .env
docker compose build
docker compose up -d
```

Each user supplies their own `LLM_GATEWAY_JWT` in `.env`. Never commit secrets.

Option B — Push to a container registry

Tag and push built images so others can pull without building:

```
# Example: Docker Hub
docker tag bodhiect-backend:latest yourorg/bodhiect-backend:0.1.0
docker tag bodhiect-frontend:latest yourorg/bodhiect-frontend:0.1.0

docker login
docker push yourorg/bodhiect-backend:0.1.0
docker push yourorg/bodhiect-frontend:0.1.0
```

Recipients use a compose file with `image:` instead of `build:` (FalkorDB can stay as official `falkordb/falkordb:latest`):

```
docker compose pull
docker compose up -d
```

Notes:

- Backend image is ~23 GB (ML/OCR deps) — first push/pull is slow.
- Use version tags (`0.1.0`) in production, not only `latest`.
- For private/org use, prefer Azure ACR, AWS ECR, or GitHub Container Registry (`ghcr.io`).

Option C — Offline export

```
docker save bodhiect-backend:latest bodhiect-frontend:latest -o bodhiect-images.tar
# recipient:
docker load -i bodhiect-images.tar
```

Environment variables

Copy `.env.example` to `.env` at the **repo root**. Loaded by `config/settings.py`; env vars override defaults.

Required for LLM features

Variable	Description
LLM_GATEWAY_BASE_URL	Gateway URL (no trailing slash)
LLM_GATEWAY_JWT or LLM_GATEWAY_TOKEN	Bearer token
LLM_GATEWAY_MODEL	e.g. <code>claude-opus-4.6</code>

FalkorDB

Variable	Default	Docker Compose
FALKORDB_HOST	<code>localhost</code>	Overridden to <code>falkordb</code>
FALKORDB_PORT	<code>6379</code>	<code>6379</code>
FALKORDB_GRAPH_NAME	<code>bodhi_graph</code>	unchanged
FALKORDB_USE MOCK	<code>false</code>	keep <code>false</code> in production

API / storage / OCR

Variable	Default	Notes
API__HOST	<code>0.0.0.0</code>	
API__PORT	<code>8000</code>	
TESSERACT_CMD	—	Windows path locally; <code>/usr/bin/tesseract</code> in Docker
STORAGE_WORKSPACE_STORE_PATH	<code>data/workspaces/workspaces.json</code>	Bind-mounted in Docker

See `.env.example` for the full list.

Verify the stack

Docker

```
docker compose ps # all healthy
curl.exe http://localhost:8080/api/workspaces # JSON workspace list
docker exec bodhi-falkordb redis-cli ping # PONG
docker compose logs backend | grep "Connected to FalkorDB" # real DB, not mock
```

Local dev

```
curl http://localhost:8000/health
curl http://localhost:8000/workspaces
```

Open the UI → create/select a workspace → **Ingestion** → **Graph** → **Retrieval** (needs LLM).

End-to-end smoke test

1. Create a workspace in the UI.
2. Upload a CSV or invoice PDF on **Ingestion**.

3. Watch pipeline progress (SSE stream on **Pipeline** page).
4. Open **Graph** to see nodes.
5. Ask a question on **Retrieval** (requires LLM gateway).

Running tests

```
uv sync
uv run pytest # full suite (~2900 tests)
uv run pytest tests/api tests/pipeline # API + pipeline subset
uv run pytest -m offline # no DB/LLM required
uv run pytest -m integration # needs live FalkorDB
```

Tests force `NER_GLINER_ENABLED=false` and `HF_HUB_OFFLINE=1` via `tests/conftest.py`.

Backend & frontend (local dev details)

Backend

```
uv sync
uv run uvicorn api.main:app --reload --host 0.0.0.0 --port 8000 # dev
uv run uvicorn api.main:app --host 0.0.0.0 --port 8000 --workers 1 # prod-style
```

Use `--workers 1` until JSON workspace stores support multi-process safety.

CLI alternative: `uv run python app.py --help`

Frontend

```
cd frontend && npm install && npm run dev # dev → :5173
cd frontend && npm run build && npm run preview # prod preview → :4173
```

Custom API URL at build time: `VITE_API_BASE_URL=https://api.example.com npm run build`

Project layout

```
BodhiECG/
├── api/ # Unified FastAPI app
├── pipeline/ # PipelineOrchestrator + stages
├── ingestion/ # Structured / unstructured / semi-structured engines
├── graph/ # FalkorDB client
├── storage/ # Workspace & KO JSON stores
├── config/ # Settings (single source of env config)
├── templates/ # Semi-structured document templates
├── data/ # Workspaces, uploads, cache (bind-mounted in Docker)
├── frontend/ # React + Vite UI
├── docker/ # Dockerfiles + nginx config
├── docker-compose.yml # Full stack
├── tests/
└── docs/
```

Common issues

Symptom	Fix
Graph empty after ingest	FalkorDB not running; check <code>FALKORDB_USE MOCK=false</code>
Backend log: <code>switching to in-memory mock</code>	FalkorDB container not on compose network — usually a port bind failure; see below
Bind for 0.0.0.0:3000 failed: port is already allocated	Another FalkorDB uses 6379/3000 — <code>docker ps</code> , stop it (<code>docker stop <name></code>), or set <code>FALKORDB_REDIS_PORT=6380</code> / <code>FALKORDB_BROWSER_PORT=3001</code>
<code>docker compose ps</code> empty or all Created	FalkorDB failed to start → whole stack aborted; fix ports, then <code>docker compose down && docker compose up -d</code>
Port 8000 already in use	Stop Docker backend or local <code>uvicorn</code> — run one mode only
NL→Cypher / retrieval 500	Set <code>LLM_GATEWAY_JWT</code> and <code>LLM_GATEWAY_BASE_URL</code> in <code>.env</code>
Image scan ingest fails (local dev)	Install Tesseract; set <code>TESSERACT_CMD</code> on Windows
Frontend API errors (local dev)	Backend must be on port 8000 (Vite proxy target)
UI not loading on Docker	Use <code>http://localhost:8080</code> , not <code>:5173</code>
<code>uv sync</code> fails on OneDrive	<code>link-mode = \"copy\"</code> already set in <code>pyproject.toml</code>
PowerShell <code>curl</code> fails	Use <code>curl.exe</code> instead of <code>curl</code>

Further reading

Document	Contents
<code>docs/StabilizationReport.md</code>	Production hardening notes
<code>api/README.md</code>	API package overview
<code>config/README.md</code>	Settings architecture
<code>frontend/src/api/client.ts</code>	How the UI talks to the API

License

Internal / enterprise use — see your organization's policy.

Appendix D — Repository Directory Tree (partial)

Generated by walking BodhiECG/ to depth 4, excluding .git, .venv, node_modules, and data/huggingface model caches.

```
.vscode/
settings.json
api/
auth/
  __init__.py
  jwt_auth.py
dependencies/
  __init__.py
providers.py
events/
  __init__.py
pipeline_events.py
lifecycle/
  __init__.py
lifespan.py
middleware/
  __init__.py
error_middleware.py
logging_middleware.py
routers/
  __init__.py
alignment.py
graph.py
ingestion.py
knowledge_objects.py
pipeline.py
retrieval.py
review.py
semi_structured.py
workspaces.py
schemas/
  __init__.py
  alignment.py
  common.py
  graph.py
  ingestion.py
  knowledge_object.py
  pipeline.py
  retrieval.py
  review.py
  semi_structured.py
  workspace.py
services/
  __init__.py
  alignment_service.py
  concept_service.py
  graph_service.py
  ingestion_service.py
  knowledge_object_service.py
  pipeline_service.py
  retrieval_service.py
  review_service.py
  semi_structured_service.py
  workspace_service.py
websocket/
  __init__.py
  __init__.py
main.py
README.md
common/
  __init__.py
base.py
canonical_pipeline.py
constants.py
exceptions.py
filesystem.py
logging.py
README.md
retry.py
serialization.py
```

```
timing.py
validation.py
config/
  __init__.py
README.md
settings.py
data/
artifacts/
medicalocr/
bills/
  med_doc_bill_100001_noisy.jpg
  med_doc_bill_100002_noisy.jpg
  med_doc_bill_100003_noisy.jpg
  med_doc_bill_100004_noisy.jpg
  med_doc_bill_100005_noisy.jpg
  med_doc_bill_100006_noisy.jpg
  med_doc_bill_100007_noisy.jpg
  med_doc_bill_100008_noisy.jpg
  med_doc_bill_100009_noisy.jpg
  med_doc_bill_100010_noisy.jpg
  med_doc_bill_100011_noisy.jpg
  med_doc_bill_100012_noisy.jpg
  med_doc_bill_100013_noisy.jpg
  med_doc_bill_100014_noisy.jpg
  med_doc_bill_100015_noisy.jpg
  med_doc_bill_100016_noisy.jpg
  med_doc_bill_100017_noisy.jpg
  med_doc_bill_100018_noisy.jpg
  med_doc_bill_100019_noisy.jpg
  med_doc_bill_100020_noisy.jpg
  med_doc_bill_100021_noisy.jpg
  med_doc_bill_100022_noisy.jpg
  med_doc_bill_100023_noisy.jpg
  med_doc_bill_100024_noisy.jpg
  med_doc_bill_100025_noisy.jpg
  med_doc_bill_100026_noisy.jpg
  med_doc_bill_100027_noisy.jpg
  med_doc_bill_100028_noisy.jpg
  med_doc_bill_100029_noisy.jpg
  med_doc_bill_100030_noisy.jpg
  med_doc_bill_100031_noisy.jpg
  med_doc_bill_100032_noisy.jpg
  med_doc_bill_100033_noisy.jpg
  med_doc_bill_100034_noisy.jpg
  med_doc_bill_100035_noisy.jpg
  med_doc_bill_100036_noisy.jpg
  med_doc_bill_100037_noisy.jpg
  med_doc_bill_100038_noisy.jpg
  med_doc_bill_100039_noisy.jpg
  med_doc_bill_100040_noisy.jpg
  med_doc_bill_100041_noisy.jpg
  med_doc_bill_100042_noisy.jpg
  med_doc_bill_100043_noisy.jpg
  med_doc_bill_100044_noisy.jpg
  med_doc_bill_100045_noisy.jpg
  med_doc_bill_100046_noisy.jpg
  med_doc_bill_100047_noisy.jpg
  med_doc_bill_100048_noisy.jpg
  med_doc_bill_100049_noisy.jpg
  med_doc_bill_100050_noisy.jpg
  med_doc_bill_100051_noisy.jpg
  med_doc_bill_100052_noisy.jpg
  med_doc_bill_100053_noisy.jpg
  med_doc_bill_100054_noisy.jpg
  med_doc_bill_100055_noisy.jpg
  med_doc_bill_100056_noisy.jpg
  med_doc_bill_100057_noisy.jpg
  med_doc_bill_100058_noisy.jpg
  med_doc_bill_100059_noisy.jpg
  med_doc_bill_100060_noisy.jpg
```

```
med_doc_bill_100061_noisy.jpg
med_doc_bill_100062_noisy.jpg
med_doc_bill_100063_noisy.jpg
med_doc_bill_100064_noisy.jpg
med_doc_bill_100065_noisy.jpg
med_doc_bill_100066_noisy.jpg
med_doc_bill_100067_noisy.jpg
med_doc_bill_100068_noisy.jpg
med_doc_bill_100069_noisy.jpg
med_doc_bill_100070_noisy.jpg
med_doc_bill_100071_noisy.jpg
med_doc_bill_100072_noisy.jpg
med_doc_bill_100073_noisy.jpg
med_doc_bill_100074_noisy.jpg
med_doc_bill_100075_noisy.jpg
med_doc_bill_100076_noisy.jpg
med_doc_bill_100077_noisy.jpg
med_doc_bill_100078_noisy.jpg
med_doc_bill_100079_noisy.jpg
med_doc_bill_100080_noisy.jpg
med_doc_bill_100081_noisy.jpg
med_doc_bill_100082_noisy.jpg
med_doc_bill_100083_noisy.jpg
med_doc_bill_100084_noisy.jpg
med_doc_bill_100085_noisy.jpg
med_doc_bill_100086_noisy.jpg
med_doc_bill_100087_noisy.jpg
med_doc_bill_100088_noisy.jpg
med_doc_bill_100089_noisy.jpg
med_doc_bill_100090_noisy.jpg
med_doc_bill_100091_noisy.jpg
med_doc_bill_100092_noisy.jpg
med_doc_bill_100093_noisy.jpg
med_doc_bill_100094_noisy.jpg
med_doc_bill_100095_noisy.jpg
med_doc_bill_100096_noisy.jpg
med_doc_bill_100097_noisy.jpg
med_doc_bill_100098_noisy.jpg
med_doc_bill_100099_noisy.jpg
med_doc_bill_100100_noisy.jpg
med_doc_bill_100101_noisy.jpg
med_doc_bill_100102_noisy.jpg
med_doc_bill_100103_noisy.jpg
med_doc_bill_100104_noisy.jpg
med_doc_bill_100105_noisy.jpg
med_doc_bill_100106_noisy.jpg
med_doc_bill_100107_noisy.jpg
med_doc_bill_100108_noisy.jpg
med_doc_bill_100109_noisy.jpg
med_doc_bill_100110_noisy.jpg
med_doc_bill_100111_noisy.jpg
med_doc_bill_100112_noisy.jpg
med_doc_bill_100113_noisy.jpg
med_doc_bill_100114_noisy.jpg
med_doc_bill_100115_noisy.jpg
med_doc_bill_100116_noisy.jpg
med_doc_bill_100117_noisy.jpg
med_doc_bill_100118_noisy.jpg
med_doc_bill_100119_noisy.jpg
med_doc_bill_100120_noisy.jpg
med_doc_bill_100121_noisy.jpg
med_doc_bill_100122_noisy.jpg
med_doc_bill_100123_noisy.jpg
med_doc_bill_100124_noisy.jpg
med_doc_bill_100125_noisy.jpg
med_doc_bill_100126_noisy.jpg
med_doc_bill_100127_noisy.jpg
med_doc_bill_100128_noisy.jpg
med_doc_bill_100129_noisy.jpg
med_doc_bill_100130_noisy.jpg
```

[illegible]

```
med_doc_bill_100451_noisy.jpg
med_doc_bill_100452_noisy.jpg
med_doc_bill_100453_noisy.jpg
med_doc_bill_100454_noisy.jpg
med_doc_bill_100455_noisy.jpg
med_doc_bill_100456_noisy.jpg
med_doc_bill_100457_noisy.jpg
med_doc_bill_100458_noisy.jpg
med_doc_bill_100459_noisy.jpg
med_doc_bill_100460_noisy.jpg
med_doc_bill_100461_noisy.jpg
med_doc_bill_100462_noisy.jpg
med_doc_bill_100463_noisy.jpg
med_doc_bill_100464_noisy.jpg
med_doc_bill_100465_noisy.jpg
med_doc_bill_100466_noisy.jpg
med_doc_bill_100467_noisy.jpg
med_doc_bill_100468_noisy.jpg
med_doc_bill_100469_noisy.jpg
med_doc_bill_100470_noisy.jpg
med_doc_bill_100471_noisy.jpg
med_doc_bill_100472_noisy.jpg
med_doc_bill_100473_noisy.jpg
med_doc_bill_100474_noisy.jpg
med_doc_bill_100475_noisy.jpg
med_doc_bill_100476_noisy.jpg
med_doc_bill_100477_noisy.jpg
med_doc_bill_100478_noisy.jpg
med_doc_bill_100479_noisy.jpg
med_doc_bill_100480_noisy.jpg
med_doc_bill_100481_noisy.jpg
med_doc_bill_100482_noisy.jpg
med_doc_bill_100483_noisy.jpg
med_doc_bill_100484_noisy.jpg
med_doc_bill_100485_noisy.jpg
med_doc_bill_100486_noisy.jpg
med_doc_bill_100487_noisy.jpg
med_doc_bill_100488_noisy.jpg
med_doc_bill_100489_noisy.jpg
med_doc_bill_100490_noisy.jpg
med_doc_bill_100491_noisy.jpg
med_doc_bill_100492_noisy.jpg
med_doc_bill_100493_noisy.jpg
med_doc_bill_100494_noisy.jpg
med_doc_bill_100495_noisy.jpg
med_doc_bill_100496_noisy.jpg
med_doc_bill_100497_noisy.jpg
med_doc_bill_100498_noisy.jpg
med_doc_bill_100499_noisy.jpg
med_doc_bill_100500_noisy.jpg
cache/
ingested.json
needs_review.json
pending.json
discharge_summaries/
med_doc_discharge_summary_2000
01_noisy.jpg
med_doc_discharge_summary_2000
02_noisy.jpg
med_doc_discharge_summary_2000
03_noisy.jpg
med_doc_discharge_summary_2000
04_noisy.jpg
med_doc_discharge_summary_2000
05_noisy.jpg
med_doc_discharge_summary_2000
06_noisy.jpg
med_doc_discharge_summary_2000
07_noisy.jpg
med_doc_discharge_summary_2000
08_noisy.jpg
med_doc_discharge_summary_2000
09_noisy.jpg
med_doc_discharge_summary_2000
10_noisy.jpg
med_doc_discharge_summary_2000
11_noisy.jpg
med_doc_discharge_summary_2000
12_noisy.jpg
med doc discharge summary 2000
```

Page 85 of 107

33_noisy.jpg
med_doc_discharge_summary_2001
34_noisy.jpg
med_doc_discharge_summary_2001
35_noisy.jpg
med_doc_discharge_summary_2001
36_noisy.jpg
med_doc_discharge_summary_2001
37_noisy.jpg
med_doc_discharge_summary_2001
38_noisy.jpg
med_doc_discharge_summary_2001
39_noisy.jpg
med_doc_discharge_summary_2001
40_noisy.jpg
med_doc_discharge_summary_2001
41_noisy.jpg
med_doc_discharge_summary_2001
42_noisy.jpg
med_doc_discharge_summary_2001
43_noisy.jpg
med_doc_discharge_summary_2001
44_noisy.jpg
med_doc_discharge_summary_2001
45_noisy.jpg
med_doc_discharge_summary_2001
46_noisy.jpg
med_doc_discharge_summary_2001
47_noisy.jpg
med_doc_discharge_summary_2001
48_noisy.jpg
med_doc_discharge_summary_2001
49_noisy.jpg
med_doc_discharge_summary_2001
50_noisy.jpg
med_doc_discharge_summary_2001
51_noisy.jpg
med_doc_discharge_summary_2001
52_noisy.jpg
med_doc_discharge_summary_2001
53_noisy.jpg
med_doc_discharge_summary_2001
54_noisy.jpg
med_doc_discharge_summary_2001
55_noisy.jpg
med_doc_discharge_summary_2001
56_noisy.jpg
med_doc_discharge_summary_2001
57_noisy.jpg
med_doc_discharge_summary_2001
58_noisy.jpg
med_doc_discharge_summary_2001
59_noisy.jpg
med_doc_discharge_summary_2001
60_noisy.jpg
med_doc_discharge_summary_2001
61_noisy.jpg
med_doc_discharge_summary_2001
62_noisy.jpg
med_doc_discharge_summary_2001
63_noisy.jpg
med_doc_discharge_summary_2001
64_noisy.jpg
med_doc_discharge_summary_2001
65_noisy.jpg
med_doc_discharge_summary_2001
66_noisy.jpg
med_doc_discharge_summary_2001
67_noisy.jpg
med_doc_discharge_summary_2001
68_noisy.jpg
med_doc_discharge_summary_2001
69_noisy.jpg
med_doc_discharge_summary_2001
70_noisy.jpg
med_doc_discharge_summary_2001
71_noisy.jpg
med_doc_discharge_summary_2001
72_noisy.jpg
med doc discharge summary 2001

53_noisy.jpg
med_doc_discharge_summary_2002
54_noisy.jpg
med_doc_discharge_summary_2002
55_noisy.jpg
med_doc_discharge_summary_2002
56_noisy.jpg
med_doc_discharge_summary_2002
57_noisy.jpg
med_doc_discharge_summary_2002
58_noisy.jpg
med_doc_discharge_summary_2002
59_noisy.jpg
med_doc_discharge_summary_2002
60_noisy.jpg
med_doc_discharge_summary_2002
61_noisy.jpg
med_doc_discharge_summary_2002
62_noisy.jpg
med_doc_discharge_summary_2002
63_noisy.jpg
med_doc_discharge_summary_2002
64_noisy.jpg
med_doc_discharge_summary_2002
65_noisy.jpg
med_doc_discharge_summary_2002
66_noisy.jpg
med_doc_discharge_summary_2002
67_noisy.jpg
med_doc_discharge_summary_2002
68_noisy.jpg
med_doc_discharge_summary_2002
69_noisy.jpg
med_doc_discharge_summary_2002
70_noisy.jpg
med_doc_discharge_summary_2002
71_noisy.jpg
med_doc_discharge_summary_2002
72_noisy.jpg
med_doc_discharge_summary_2002
73_noisy.jpg
med_doc_discharge_summary_2002
74_noisy.jpg
med_doc_discharge_summary_2002
75_noisy.jpg
med_doc_discharge_summary_2002
76_noisy.jpg
med_doc_discharge_summary_2002
77_noisy.jpg
med_doc_discharge_summary_2002
78_noisy.jpg
med_doc_discharge_summary_2002
79_noisy.jpg
med_doc_discharge_summary_2002
80_noisy.jpg
med_doc_discharge_summary_2002
81_noisy.jpg
med_doc_discharge_summary_2002
82_noisy.jpg
med_doc_discharge_summary_2002
83_noisy.jpg
med_doc_discharge_summary_2002
84_noisy.jpg
med_doc_discharge_summary_2002
85_noisy.jpg
med_doc_discharge_summary_2002
86_noisy.jpg
med_doc_discharge_summary_2002
87_noisy.jpg
med_doc_discharge_summary_2002
88_noisy.jpg
med_doc_discharge_summary_2002
89_noisy.jpg
med_doc_discharge_summary_2002
90_noisy.jpg
med_doc_discharge_summary_2002
91_noisy.jpg
med_doc_discharge_summary_2002
92_noisy.jpg
med doc discharge summary 2002

73_noisy.jpg
med_doc_discharge_summary_2003
74_noisy.jpg
med_doc_discharge_summary_2003
75_noisy.jpg
med_doc_discharge_summary_2003
76_noisy.jpg
med_doc_discharge_summary_2003
77_noisy.jpg
med_doc_discharge_summary_2003
78_noisy.jpg
med_doc_discharge_summary_2003
79_noisy.jpg
med_doc_discharge_summary_2003
80_noisy.jpg
med_doc_discharge_summary_2003
81_noisy.jpg
med_doc_discharge_summary_2003
82_noisy.jpg
med_doc_discharge_summary_2003
83_noisy.jpg
med_doc_discharge_summary_2003
84_noisy.jpg
med_doc_discharge_summary_2003
85_noisy.jpg
med_doc_discharge_summary_2003
86_noisy.jpg
med_doc_discharge_summary_2003
87_noisy.jpg
med_doc_discharge_summary_2003
88_noisy.jpg
med_doc_discharge_summary_2003
89_noisy.jpg
med_doc_discharge_summary_2003
90_noisy.jpg
med_doc_discharge_summary_2003
91_noisy.jpg
med_doc_discharge_summary_2003
92_noisy.jpg
med_doc_discharge_summary_2003
93_noisy.jpg
med_doc_discharge_summary_2003
94_noisy.jpg
med_doc_discharge_summary_2003
95_noisy.jpg
med_doc_discharge_summary_2003
96_noisy.jpg
med_doc_discharge_summary_2003
97_noisy.jpg
med_doc_discharge_summary_2003
98_noisy.jpg
med_doc_discharge_summary_2003
99_noisy.jpg
med_doc_discharge_summary_2004
00_noisy.jpg
med_doc_discharge_summary_2004
01_noisy.jpg
med_doc_discharge_summary_2004
02_noisy.jpg
med_doc_discharge_summary_2004
03_noisy.jpg
med_doc_discharge_summary_2004
04_noisy.jpg
med_doc_discharge_summary_2004
05_noisy.jpg
med_doc_discharge_summary_2004
06_noisy.jpg
med_doc_discharge_summary_2004
07_noisy.jpg
med_doc_discharge_summary_2004
08_noisy.jpg
med_doc_discharge_summary_2004
09_noisy.jpg
med_doc_discharge_summary_2004
10_noisy.jpg
med_doc_discharge_summary_2004
11_noisy.jpg
med_doc_discharge_summary_2004
12_noisy.jpg
med doc discharge summary 2004

13_noisy.jpg
med_doc_discharge_summary_2004
14_noisy.jpg
med_doc_discharge_summary_2004
15_noisy.jpg
med_doc_discharge_summary_2004
16_noisy.jpg
med_doc_discharge_summary_2004
17_noisy.jpg
med_doc_discharge_summary_2004
18_noisy.jpg
med_doc_discharge_summary_2004
19_noisy.jpg
med_doc_discharge_summary_2004
20_noisy.jpg
med_doc_discharge_summary_2004
21_noisy.jpg
med_doc_discharge_summary_2004
22_noisy.jpg
med_doc_discharge_summary_2004
23_noisy.jpg
med_doc_discharge_summary_2004
24_noisy.jpg
med_doc_discharge_summary_2004
25_noisy.jpg
med_doc_discharge_summary_2004
26_noisy.jpg
med_doc_discharge_summary_2004
27_noisy.jpg
med_doc_discharge_summary_2004
28_noisy.jpg
med_doc_discharge_summary_2004
29_noisy.jpg
med_doc_discharge_summary_2004
30_noisy.jpg
med_doc_discharge_summary_2004
31_noisy.jpg
med_doc_discharge_summary_2004
32_noisy.jpg
med_doc_discharge_summary_2004
33_noisy.jpg
med_doc_discharge_summary_2004
34_noisy.jpg
med_doc_discharge_summary_2004
35_noisy.jpg
med_doc_discharge_summary_2004
36_noisy.jpg
med_doc_discharge_summary_2004
37_noisy.jpg
med_doc_discharge_summary_2004
38_noisy.jpg
med_doc_discharge_summary_2004
39_noisy.jpg
med_doc_discharge_summary_2004
40_noisy.jpg
med_doc_discharge_summary_2004
41_noisy.jpg
med_doc_discharge_summary_2004
42_noisy.jpg
med_doc_discharge_summary_2004
43_noisy.jpg
med_doc_discharge_summary_2004
44_noisy.jpg
med_doc_discharge_summary_2004
45_noisy.jpg
med_doc_discharge_summary_2004
46_noisy.jpg
med_doc_discharge_summary_2004
47_noisy.jpg
med_doc_discharge_summary_2004
48_noisy.jpg
med_doc_discharge_summary_2004
49_noisy.jpg
med_doc_discharge_summary_2004
50_noisy.jpg
med_doc_discharge_summary_2004
51_noisy.jpg
med_doc_discharge_summary_2004
52_noisy.jpg
med_doc_discharge_summary_2004

53_noisy.jpg
med_doc_discharge_summary_2004
54_noisy.jpg
med_doc_discharge_summary_2004
55_noisy.jpg
med_doc_discharge_summary_2004
56_noisy.jpg
med_doc_discharge_summary_2004
57_noisy.jpg
med_doc_discharge_summary_2004
58_noisy.jpg
med_doc_discharge_summary_2004
59_noisy.jpg
med_doc_discharge_summary_2004
60_noisy.jpg
med_doc_discharge_summary_2004
61_noisy.jpg
med_doc_discharge_summary_2004
62_noisy.jpg
med_doc_discharge_summary_2004
63_noisy.jpg
med_doc_discharge_summary_2004
64_noisy.jpg
med_doc_discharge_summary_2004
65_noisy.jpg
med_doc_discharge_summary_2004
66_noisy.jpg
med_doc_discharge_summary_2004
67_noisy.jpg
med_doc_discharge_summary_2004
68_noisy.jpg
med_doc_discharge_summary_2004
69_noisy.jpg
med_doc_discharge_summary_2004
70_noisy.jpg
med_doc_discharge_summary_2004
71_noisy.jpg
med_doc_discharge_summary_2004
72_noisy.jpg
med_doc_discharge_summary_2004
73_noisy.jpg
med_doc_discharge_summary_2004
74_noisy.jpg
med_doc_discharge_summary_2004
75_noisy.jpg
med_doc_discharge_summary_2004
76_noisy.jpg
med_doc_discharge_summary_2004
77_noisy.jpg
med_doc_discharge_summary_2004
78_noisy.jpg
med_doc_discharge_summary_2004
79_noisy.jpg
med_doc_discharge_summary_2004
80_noisy.jpg
med_doc_discharge_summary_2004
81_noisy.jpg
med_doc_discharge_summary_2004
82_noisy.jpg
med_doc_discharge_summary_2004
83_noisy.jpg
med_doc_discharge_summary_2004
84_noisy.jpg
med_doc_discharge_summary_2004
85_noisy.jpg
med_doc_discharge_summary_2004
86_noisy.jpg
med_doc_discharge_summary_2004
87_noisy.jpg
med_doc_discharge_summary_2004
88_noisy.jpg
med_doc_discharge_summary_2004
89_noisy.jpg
med_doc_discharge_summary_2004
90_noisy.jpg
med_doc_discharge_summary_2004
91_noisy.jpg
med_doc_discharge_summary_2004
92_noisy.jpg
med_doc_discharge_summary_2004

93_noisy.jpg
med_doc_discharge_summary_2004
94_noisy.jpg
med_doc_discharge_summary_2004
95_noisy.jpg
med_doc_discharge_summary_2004
96_noisy.jpg
med_doc_discharge_summary_2004
97_noisy.jpg
med_doc_discharge_summary_2004
98_noisy.jpg
med_doc_discharge_summary_2004
99_noisy.jpg
med_doc_discharge_summary_2005
00_noisy.jpg
output/
medicalocr.csv
ontology/
finance_advisory_ontology_v2
2.owl
telecom-churn (2).owl
semi_structured/
cache/
ingested.json
needs_review.json
pending.json
ontology/
combined.owl
discharge_summary.owl
medical_bill.owl
uploads/
invoice_51109301.pdf
invoice_51109302.pdf
invoice_51109303.pdf
invoice_51109304.pdf
invoice_51109305.pdf
invoice_51109306.pdf
invoice_51109307.pdf
invoice_51109308.pdf
invoice_51109309.pdf
invoice_51109310.pdf
invoice_51109311.pdf
invoice_51109312.pdf
invoice_51109313.pdf
invoice_51109314.pdf
invoice_51109315.pdf
invoice_51109316.pdf
invoice_51109317.pdf
invoice_51109318.pdf
invoice_51109319.pdf
invoice_51109320.pdf
invoice_51109321.pdf
invoice_51109322.pdf
invoice_51109323.pdf
invoice_51109324.pdf
invoice_51109325.pdf
invoice_51109326.pdf
invoice_51109327.pdf
invoice_51109328.pdf
invoice_51109329.pdf
invoice_51109330.pdf
invoice_51109331.pdf
invoice_51109332.pdf
invoice_51109333.pdf
invoice_51109334.pdf
invoice_51109335.pdf
invoice_51109336.pdf
invoice_51109337.pdf
invoice_51109338.pdf
invoice_51109339.pdf
invoice_51109340.pdf
invoice_51109341.pdf
invoice_51109342.pdf
invoice_51109343.pdf
invoice_51109344.pdf
invoice_51109345.pdf
invoice_51109346.pdf
invoice_51109347.pdf
invoice_51109348.pdf
invoice_51109349.pdf

[illegible]

[illegible]

45_noisy.jpg
med_doc_discharge_summary_2000
46_noisy.jpg
med_doc_discharge_summary_2000
47_noisy.jpg
med_doc_discharge_summary_2000
48_noisy.jpg
med_doc_discharge_summary_2000
49_noisy.jpg
med_doc_discharge_summary_2000
50_noisy.jpg
med_doc_discharge_summary_2000
51_noisy.jpg
med_doc_discharge_summary_2000
52_noisy.jpg
med_doc_discharge_summary_2000
53_noisy.jpg
med_doc_discharge_summary_2000
54_noisy.jpg
med_doc_discharge_summary_2000
55_noisy.jpg
med_doc_discharge_summary_2000
56_noisy.jpg
med_doc_discharge_summary_2000
57_noisy.jpg
med_doc_discharge_summary_2000
58_noisy.jpg
med_doc_discharge_summary_2000
59_noisy.jpg
med_doc_discharge_summary_2000
60_noisy.jpg
med_doc_discharge_summary_2000
61_noisy.jpg
med_doc_discharge_summary_2000
62_noisy.jpg
med_doc_discharge_summary_2000
63_noisy.jpg
med_doc_discharge_summary_2000
64_noisy.jpg
med_doc_discharge_summary_2000
65_noisy.jpg
med_doc_discharge_summary_2000
66_noisy.jpg
med_doc_discharge_summary_2000
67_noisy.jpg
med_doc_discharge_summary_2000
68_noisy.jpg
med_doc_discharge_summary_2000
69_noisy.jpg
med_doc_discharge_summary_2000
70_noisy.jpg
med_doc_discharge_summary_2000
71_noisy.jpg
med_doc_discharge_summary_2000
72_noisy.jpg
med_doc_discharge_summary_2000
73_noisy.jpg
med_doc_discharge_summary_2000
74_noisy.jpg
med_doc_discharge_summary_2000
75_noisy.jpg
med_doc_discharge_summary_2000
76_noisy.jpg
med_doc_discharge_summary_2000
77_noisy.jpg
med_doc_discharge_summary_2000
78_noisy.jpg
med_doc_discharge_summary_2000
79_noisy.jpg
med_doc_discharge_summary_2000
80_noisy.jpg
med_doc_discharge_summary_2000
81_noisy.jpg
med_doc_discharge_summary_2000
82_noisy.jpg
med_doc_discharge_summary_2000
83_noisy.jpg
med_doc_discharge_summary_2000
84_noisy.jpg
med doc discharge summary 2000

65_noisy.jpg
med_doc_discharge_summary_2001
66_noisy.jpg
med_doc_discharge_summary_2001
67_noisy.jpg
med_doc_discharge_summary_2001
68_noisy.jpg
med_doc_discharge_summary_2001
69_noisy.jpg
med_doc_discharge_summary_2001
70_noisy.jpg
med_doc_discharge_summary_2001
71_noisy.jpg
med_doc_discharge_summary_2001
72_noisy.jpg
med_doc_discharge_summary_2001
73_noisy.jpg
med_doc_discharge_summary_2001
74_noisy.jpg
med_doc_discharge_summary_2001
75_noisy.jpg
med_doc_discharge_summary_2001
76_noisy.jpg
med_doc_discharge_summary_2001
77_noisy.jpg
med_doc_discharge_summary_2001
78_noisy.jpg
med_doc_discharge_summary_2001
79_noisy.jpg
med_doc_discharge_summary_2001
80_noisy.jpg
med_doc_discharge_summary_2001
81_noisy.jpg
med_doc_discharge_summary_2001
82_noisy.jpg
med_doc_discharge_summary_2001
83_noisy.jpg
med_doc_discharge_summary_2001
84_noisy.jpg
med_doc_discharge_summary_2001
85_noisy.jpg
med_doc_discharge_summary_2001
86_noisy.jpg
med_doc_discharge_summary_2001
87_noisy.jpg
med_doc_discharge_summary_2001
88_noisy.jpg
med_doc_discharge_summary_2001
89_noisy.jpg
med_doc_discharge_summary_2001
90_noisy.jpg
med_doc_discharge_summary_2001
91_noisy.jpg
med_doc_discharge_summary_2001
92_noisy.jpg
med_doc_discharge_summary_2001
93_noisy.jpg
med_doc_discharge_summary_2001
94_noisy.jpg
med_doc_discharge_summary_2001
95_noisy.jpg
med_doc_discharge_summary_2001
96_noisy.jpg
med_doc_discharge_summary_2001
97_noisy.jpg
med_doc_discharge_summary_2001
98_noisy.jpg
med_doc_discharge_summary_2001
99_noisy.jpg
med_doc_discharge_summary_2002
00_noisy.jpg
med_doc_discharge_summary_2002
01_noisy.jpg
med_doc_discharge_summary_2002
02_noisy.jpg
med_doc_discharge_summary_2002
03_noisy.jpg
med_doc_discharge_summary_2002
04_noisy.jpg
med doc discharge summary 2002

85_noisy.jpg
med_doc_discharge_summary_2002
86_noisy.jpg
med_doc_discharge_summary_2002
87_noisy.jpg
med_doc_discharge_summary_2002
88_noisy.jpg
med_doc_discharge_summary_2002
89_noisy.jpg
med_doc_discharge_summary_2002
90_noisy.jpg
med_doc_discharge_summary_2002
91_noisy.jpg
med_doc_discharge_summary_2002
92_noisy.jpg
med_doc_discharge_summary_2002
93_noisy.jpg
med_doc_discharge_summary_2002
94_noisy.jpg
med_doc_discharge_summary_2002
95_noisy.jpg
med_doc_discharge_summary_2002
96_noisy.jpg
med_doc_discharge_summary_2002
97_noisy.jpg
med_doc_discharge_summary_2002
98_noisy.jpg
med_doc_discharge_summary_2002
99_noisy.jpg
med_doc_discharge_summary_2003
00_noisy.jpg
med_doc_discharge_summary_2003
01_noisy.jpg
med_doc_discharge_summary_2003
02_noisy.jpg
med_doc_discharge_summary_2003
03_noisy.jpg
med_doc_discharge_summary_2003
04_noisy.jpg
med_doc_discharge_summary_2003
05_noisy.jpg
med_doc_discharge_summary_2003
06_noisy.jpg
med_doc_discharge_summary_2003
07_noisy.jpg
med_doc_discharge_summary_2003
08_noisy.jpg
med_doc_discharge_summary_2003
09_noisy.jpg
med_doc_discharge_summary_2003
10_noisy.jpg
med_doc_discharge_summary_2003
11_noisy.jpg
med_doc_discharge_summary_2003
12_noisy.jpg
med_doc_discharge_summary_2003
13_noisy.jpg
med_doc_discharge_summary_2003
14_noisy.jpg
med_doc_discharge_summary_2003
15_noisy.jpg
med_doc_discharge_summary_2003
16_noisy.jpg
med_doc_discharge_summary_2003
17_noisy.jpg
med_doc_discharge_summary_2003
18_noisy.jpg
med_doc_discharge_summary_2003
19_noisy.jpg
med_doc_discharge_summary_2003
20_noisy.jpg
med_doc_discharge_summary_2003
21_noisy.jpg
med_doc_discharge_summary_2003
22_noisy.jpg
med_doc_discharge_summary_2003
23_noisy.jpg
med_doc_discharge_summary_2003
24_noisy.jpg
med doc discharge summary 2003

05_noisy.jpg
med_doc_discharge_summary_2004
06_noisy.jpg
med_doc_discharge_summary_2004
07_noisy.jpg
med_doc_discharge_summary_2004
08_noisy.jpg
med_doc_discharge_summary_2004
09_noisy.jpg
med_doc_discharge_summary_2004
10_noisy.jpg
med_doc_discharge_summary_2004
11_noisy.jpg
med_doc_discharge_summary_2004
12_noisy.jpg
med_doc_discharge_summary_2004
13_noisy.jpg
med_doc_discharge_summary_2004
14_noisy.jpg
med_doc_discharge_summary_2004
15_noisy.jpg
med_doc_discharge_summary_2004
16_noisy.jpg
med_doc_discharge_summary_2004
17_noisy.jpg
med_doc_discharge_summary_2004
18_noisy.jpg
med_doc_discharge_summary_2004
19_noisy.jpg
med_doc_discharge_summary_2004
20_noisy.jpg
med_doc_discharge_summary_2004
21_noisy.jpg
med_doc_discharge_summary_2004
22_noisy.jpg
med_doc_discharge_summary_2004
23_noisy.jpg
med_doc_discharge_summary_2004
24_noisy.jpg
med_doc_discharge_summary_2004
25_noisy.jpg
med_doc_discharge_summary_2004
26_noisy.jpg
med_doc_discharge_summary_2004
27_noisy.jpg
med_doc_discharge_summary_2004
28_noisy.jpg
med_doc_discharge_summary_2004
29_noisy.jpg
med_doc_discharge_summary_2004
30_noisy.jpg
med_doc_discharge_summary_2004
31_noisy.jpg
med_doc_discharge_summary_2004
32_noisy.jpg
med_doc_discharge_summary_2004
33_noisy.jpg
med_doc_discharge_summary_2004
34_noisy.jpg
med_doc_discharge_summary_2004
35_noisy.jpg
med_doc_discharge_summary_2004
36_noisy.jpg
med_doc_discharge_summary_2004
37_noisy.jpg
med_doc_discharge_summary_2004
38_noisy.jpg
med_doc_discharge_summary_2004
39_noisy.jpg
med_doc_discharge_summary_2004
40_noisy.jpg
med_doc_discharge_summary_2004
41_noisy.jpg
med_doc_discharge_summary_2004
42_noisy.jpg
med_doc_discharge_summary_2004
43_noisy.jpg
med_doc_discharge_summary_2004
44_noisy.jpg
med doc discharge summary 2004

45_noisy.jpg
 med_doc_discharge_summary_2004
 46_noisy.jpg
 med_doc_discharge_summary_2004
 47_noisy.jpg
 med_doc_discharge_summary_2004
 48_noisy.jpg
 med_doc_discharge_summary_2004
 49_noisy.jpg
 med_doc_discharge_summary_2004
 50_noisy.jpg
 med_doc_discharge_summary_2004
 51_noisy.jpg
 med_doc_discharge_summary_2004
 52_noisy.jpg
 med_doc_discharge_summary_2004
 53_noisy.jpg
 med_doc_discharge_summary_2004
 54_noisy.jpg
 med_doc_discharge_summary_2004
 55_noisy.jpg
 med_doc_discharge_summary_2004
 56_noisy.jpg
 med_doc_discharge_summary_2004
 57_noisy.jpg
 med_doc_discharge_summary_2004
 58_noisy.jpg
 med_doc_discharge_summary_2004
 59_noisy.jpg
 med_doc_discharge_summary_2004
 60_noisy.jpg
 med_doc_discharge_summary_2004
 61_noisy.jpg
 med_doc_discharge_summary_2004
 62_noisy.jpg
 med_doc_discharge_summary_2004
 63_noisy.jpg
 med_doc_discharge_summary_2004
 64_noisy.jpg
 med_doc_discharge_summary_2004
 65_noisy.jpg
 med_doc_discharge_summary_2004
 66_noisy.jpg
 med_doc_discharge_summary_2004
 67_noisy.jpg
 med_doc_discharge_summary_2004
 68_noisy.jpg
 med_doc_discharge_summary_2004
 69_noisy.jpg
 med_doc_discharge_summary_2004
 70_noisy.jpg
 med_doc_discharge_summary_2004
 71_noisy.jpg
 med_doc_discharge_summary_2004
 72_noisy.jpg
 med_doc_discharge_summary_2004
 73_noisy.jpg
 med_doc_discharge_summary_2004
 74_noisy.jpg
 med_doc_discharge_summary_2004
 75_noisy.jpg
 med_doc_discharge_summary_2004
 76_noisy.jpg
 med_doc_discharge_summary_2004
 77_noisy.jpg
 med_doc_discharge_summary_2004
 78_noisy.jpg
 med_doc_discharge_summary_2004
 79_noisy.jpg
 med_doc_discharge_summary_2004
 80_noisy.jpg
 med_doc_discharge_summary_2004
 81_noisy.jpg
 med_doc_discharge_summary_2004
 82_noisy.jpg
 med_doc_discharge_summary_2004
 83_noisy.jpg
 med_doc_discharge_summary_2004
 84_noisy.jpg
 med_doc_discharge_summary_2004

85_noisy.jpg
 med_doc_discharge_summary_2004
 86_noisy.jpg
 med_doc_discharge_summary_2004
 87_noisy.jpg
 med_doc_discharge_summary_2004
 88_noisy.jpg
 med_doc_discharge_summary_2004
 89_noisy.jpg
 med_doc_discharge_summary_2004
 90_noisy.jpg
 med_doc_discharge_summary_2004
 91_noisy.jpg
 med_doc_discharge_summary_2004
 92_noisy.jpg
 med_doc_discharge_summary_2004
 93_noisy.jpg
 med_doc_discharge_summary_2004
 94_noisy.jpg
 med_doc_discharge_summary_2004
 95_noisy.jpg
 med_doc_discharge_summary_2004
 96_noisy.jpg
 med_doc_discharge_summary_2004
 97_noisy.jpg
 med_doc_discharge_summary_2004
 98_noisy.jpg
 med_doc_discharge_summary_2004
 99_noisy.jpg
 med_doc_discharge_summary_2005
 00_noisy.jpg
 documents.csv
 tmp/
 uploads/
 12b900ec-7bc7-46c0-ac96-
 0bbb2b2e6ec1/
 churn_risk_profile_enriched
 (1).csv
 enriched_telecom_churn (1).csv
 interventions_enriched (1).csv
 1f7c2bfe-f9c5-4028-942d-
 5a760c013552/
 advisory_sessions 1.csv
 clients 1.csv
 financial_goals 1.csv
 40366f30-8d09-42b2-958c-
 c15f42994639/
 advisory_sessions 1.csv
 clients 1.csv
 financial_goals 1.csv
 40a03534-92ab-45ee-bf5e-
 a7ec7516f262/
 advisory_sessions 1.csv
 clients 1.csv
 financial_goals 1.csv
 4289e9e7-9b4e-4455-87a5-
 aa548ee89f7a/
 advisory_sessions 1.csv
 clients 1.csv
 financial_goals 1.csv
 42b8ebde-1e05-47e1-9474-
 3627eladff8b/
 advisory_sessions 1.csv
 clients 1.csv
 financial_goals 1.csv
 46461428-af6f-4885-b856-
 b8bbfbd3824d/
 advisory_sessions 1.csv
 clients 1.csv
 financial_goals 1.csv
 49db4a6c-84e2-488d-a390-
 e763f61d42be/
 advisory_sessions 1.csv
 clients 1.csv
 financial_goals 1.csv
 4e867f34-af03-4c7f-a5b6-
 735a5d3e7042/
 med_doc_bill_100438_noisy.jpg
 594711de-fe90-47ef-a853-
 be837f5a91e2/

advisory_sessions 1.csv
 clients 1.csv
 financial_goals 1.csv
 5db62794-c637-427d-880b-
 8935213c853f/
 advisory_sessions 1.csv
 clients 1.csv
 financial_goals 1.csv
 7346ac9f-7fdd-4cb8-ac4c-
 e261145eec27/
 med_doc_discharge_summary_2004
 76_noisy.jpg
 88dd11df-3237-4374-97dd-
 abbb44920ad6/
 churn_risk_profile_enriched
 (1).csv
 enriched_telecom_churn (1).csv
 interventions_enriched (1).csv
 8bc5b203-66af-4f37-81ba-
 ebcfef9fe482/
 medical.json
 a4elb6fd-ec69-4510-9075-
 ef853a74aaed/
 invoice_51109394.pdf
 SparshVaani_IEEE_Research_Pape
 r.pdf
 a920a74b-ba10-4616-b05f-
 cccb0fe5f5f/
 medical.json
 ae7e546f-15bf-49df-a024-
 7bfcecc26faf/
 medical.json
 ca3fed60-b814-43cb-8651-
 49d51f1fb590/
 advisory_sessions 1.csv
 clients 1.csv
 financial_goals 1.csv
 cb8325c0-a26b-45e8-a9f9-
 5d9cfa869c1f/
 advisory_sessions 1.csv
 clients 1.csv
 financial_goals 1.csv
 ffdc0bda-02a5-4dc2-80ce-
 43611a3e1a42/
 advisory_sessions 1.csv
 clients 1.csv
 financial_goals 1.csv
 a4elb6fd-ec69-4510-9075-
 ef853a74aaed.json
 wsl.json
 entity_registry/
 07ab5fb4-6f77-48b2-9e09-
 474c9e8a4cd5.json
 3c129952-2f9b-443d-a1b6-
 7cde33a71f2b.json
 42b8ebde-1e05-47e1-9474-
 3627eladff8b.json
 7d32f1cb-1a3a-4d53-bd39-
 614697955e72.json
 7f578e72-72b1-467c-bae1-
 23b0d3063b4c.json
 8bc5b203-66af-4f37-81ba-
 ebcfef9fe482.json
 be6b4317-3090-4a4e-94a2-
 f5ff39098fb5.json
 c64dfec2-7c55-4ef2-8d78-
 1830961f6617.json
 c886d740-310a-42da-b2cd-
 71cd61658a6c.json
 knowledge_objects/
 5b55ead9-14ba-42c6-b9b8-
 c25f1bcc4c49.json
 a4elb6fd-ec69-4510-9075-
 ef853a74aaed.json
 retrieval_history/
 history.json

```

semantic_memory/
5b55ead9-14ba-42c6-b9b8-
c25f1bcc4c49.json
a4e1b6fd-ec69-4510-9075-
ef853a74aaed.json
workspaces.json
1_Client_Advisory_Transcripts.
pdf
2_Retirement_Policy_Document.p
df
3_Phone_Contact_Workflow.pdf
advisory_sessions 1.csv
churn_risk_profile_enriched
(1).csv
clients 1.csv
enriched_telecom_churn (1).csv
financial_goals 1.csv
interventions_enriched (1).csv
medical.json
novel.json
sample_medical.txt
docker/
backend/
Dockerfile
entrypoint.sh
prefetch_models.py
falkordb/
Dockerfile
frontend/
Dockerfile
nginx.conf
docs/
API_REFERENCE.md
APIArchitecture.md
APIContract.md
approaches-A-vs-B.md
ArchitectureReview.md
BodhiECG_Technical_Documentati
on.docx
BugReport.md
CategoryBAnalysis.md
CLIArchitecture.md
CODEBASE.md
CodeQualityReport.md
context-graph-strategy.md
ECG.drawio.svg
fable5-research-brief.md
FrontendArchitecture.md
FrontendStateManagement.md
hybrid-grounded-retrieval.md
incremental-entity-linking.md
IntegrationReport.md
KnowledgeGraphSemanticValidati
on.md
KnowledgeGraphValidation.md
KnowledgeObjectArchitecture.md
merge-plan.md
MigrationPlan.md
okf-and-hybrid-retrieval.md
performance_report.md
PerformanceReport.md
Phase2Migration.md
README.md
RealtimePipeline.md
RepositoryAudit.md
SESSION-HANDOFF.md
SharedCapabilityAudit.md
SharedExtractionReport.md
skos-concept-linking.md
spider-benchmark-results.md
StabilizationReport.md
StructuredKnowledgeObjectImple
mentation.md
StructuredMigrationReport.md
TechnicalDebt.md
TestStrategy.md
UIComponentHierarchy.md
UnifiedPipelineArchitecture.md
UnstructuredKnowledgeObjectImp
lementation.md

```

```

UnstructuredMigrationReport.md
entity_resolution/
__init__.py
base.py
README.md
frontend/
dist/
assets/
index-BXktNufs.css
index-C4-HRmMj.js
favicon.svg
icons.svg
index.html
public/
favicon.svg
icons.svg
src/
api/
alignment.ts
client.ts
graph.ts
ingestion.ts
knowledgeObjects.ts
pipeline.ts
retrieval.ts
review.ts
semiStructured.ts
workspaces.ts
assets/
components/
ui/
badge.tsx
button.tsx
card.tsx
checkbox.tsx
dialog.tsx
input.tsx
progress.tsx
select.tsx
separator.tsx
table.tsx
tabs.tsx
toast.tsx
ConfirmDialog.tsx
Markdown.tsx
QueryErrorBanner.tsx
Toaster.tsx
WorkspaceHydrator.tsx
features/
dev/
DevJson.tsx
DevPanel.tsx
graph/
CytoscapeCanvas.tsx
ingestion/
Dropzone.tsx
ontologyGraph.ts
OntologyGraphPreview.tsx
ScanIngestDropzone.tsx
ScannedDocumentsPanel.tsx
TemplateAuthorWizard.tsx
TemplateEditWizard.tsx
templateGraph.ts
TemplateGraphEditor.tsx
templateGraphMutations.ts
TemplateGraphPreview.tsx
pipeline/
retrieval/
cypherResultFormat.tsx
review/
workspace/
hooks/
useAlignment.ts
useGraph.ts
useIngestion.ts
useKnowledgeObjects.ts
usePipeline.ts
usePipelineStream.ts
useRetrieval.ts
useReview.ts

```

```

useSemiStructured.ts
useWorkspaces.ts
layouts/
AppLayout.tsx
Sidebar.tsx
Topbar.tsx
lib/
useDebounceValue.ts
utils.ts
pages/
AlignmentPage.tsx
GraphPage.tsx
IngestionPage.tsx
KnowledgeObjectsPage.tsx
PipelinePage.tsx
RetrievalPage.tsx
ReviewPage.tsx
WorkspacePage.tsx
routes/
index.tsx
store/
devStore.ts
themeStore.ts
toastStore.ts
workspaceStore.ts
types/
api.ts
App.tsx
index.css
main.tsx
.gitignore
.oxlintc.json
index.html
package-lock.json
package.json
README.md
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
graph/
__init__.py
base.py
cypher_normalize.py
falkordb.py
label_resolver.py
metrics.py
mock.py
README.md
graph_builder/
__init__.py
base.py
canonical.py
README.md
hitl/
__init__.py
README.md
ingestion/
semi_structured/
orchestration/
__init__.py
langgraph_pipeline.py
__init__.py
business_schema.py
document_cache.py
document_templates.py
entity_identity.py
entity_materializer.py
knowledge_object_builder.py
kv_extraction.py
ocr.py
owl_export.py
template_author.py
template_editor.py
template_extractor.py
template_ko_builder.py
template_storage.py
structured/
api/
schemas/

```

```

__init__.py
graph.py
ingestion.py
metrics.py
provenance.py
vl/
  __init__.py
analytics.py
business_schema.py
community.py
cypher.py
data_dictionary.py
entity.py
entity_resolution.py
export.py
graph.py
graph_import.py
graph_views.py
health.py
ingest.py
metrics.py
ner.py
ontology.py
pipeline.py
postgres.py
provenance.py
query.py
relationship.py
relationship_discovery.py
retrieval.py
review.py
router.py
schema.py
semantic_schema_api.py
session.py
system_status.py
workspace_dashboard.py
workspaces.py
  __init__.py
dependencies.py
job_store.py
main.py
middleware.py
chat/
  __init__.py
chatbot.py
cypher_safety.py
nl_to_cypher.py
query_engine.py
schema_introspector.py
chunking/
  __init__.py
base.py
hierarchical.py
models.py
orchestrator.py
recursive.py
structure.py
domain/
  __init__.py
entities.py
exceptions.py
ontology_entities.py
value_objects.py
entity_resolution/
  __init__.py
canonical.py
merge.py
metrics.py
models.py
service.py
strategies.py
type_aware_resolver.py
export/
  __init__.py
chunk_exporter.py
graph/
  __init__.py
analytics.py
community_detection.py

```

```

legacy_shim.py
segment_classifier.py
semantic_enricher.py
infrastructure/
  chunking/
    __init__.py
  service.py
graph/
  __init__.py
falkordb_adapter.py
mock_adapter.py
metrics/
  __init__.py
json_collector.py
ner/
  __init__.py
noop.py
service.py
ontology/
  __init__.py
discovery.py
domain_detector.py
mapper.py
noop.py
registry.py
review.py
parsers/
  __init__.py
registry.py
persistence/
  __init__.py
file_store.py
pii/
  __init__.py
noop.py
redactor.py
  __init__.py
ingestion/
  postgres/
    __init__.py
  adapter.py
connector.py
entity_mapper.py
metrics.py
relationship_mapper.py
schema_discovery.py
table_reader.py
  __init__.py
pipeline.py
knowledge_object/
  __init__.py
builder.py
column_classifier.py
concept_detector.py
foreign_key.py
human_review.py
ontology_parser.py
primary_key.py
profiling.py
relationship_reconciler.py
relationship_validator.py
schema_builder.py
semantic_validator.py
legacy_extraction/
  __init__.py
chunk_profiler.py
config.py
entity_extractor.py
extraction_metrics.py
normalizer.py
ontology_context_resolver.py
ontology_triple_validator.py
parser.py
pipeline.py
prompt_registry.py
py.typed
rebel_extractor.py
reference_detector.py
relationship_extractor.py
sampler.py

```

```

schema_inferer.py
triple_builder.py
validator.py
llm_orchestration/
  __init__.py
chunk_analysis.py
config.py
context_window_builder.py
entity_density_analyzer.py
gliner_adapter.py
orchestration_metrics.py
orchestrator.py
semantic_region.py
semantic_region_builder.py
metrics/
  __init__.py
collector.py
ner/
  __init__.py
base.py
config.py
fusion.py
gliner_engine.py
hybrid_engine.py
ontology_labels.py
spacy_extractor.py
observability/
  __init__.py
profiler.py
ontology/
  __init__.py
business_execution_schema.py
business_fact_service.py
business_understanding_service
.py
class_classifier.py
client_relationship_deriver.py
cluster_service.py
concept_normalizer.py
dataset_analyzer.py
deterministic_schema_generator
.py
execution_planner.py
execution_schema.py
graph_validator.py
loader.py
models.py
relationship_discoverer.py
relationship_planner.py
schema_mapper.py
semantic_analyzer.py
semantic_schema.py
typed_entity_builder.py
validator.py
orchestration/
  __init__.py
langgraph_pipeline.py
parsers/
  __init__.py
base.py
csv.py
docx.py
image.py
json.py
ontology.py
pdf.py
pptx.py
sql.py
text.py
xlsx.py
pipeline/
  __init__.py
deterministic_graph_builder.py
master.py
multi_pdf_pipeline.py
node_builder.py
ontology_csv_ingester.py
relationship_builder.py
session_pipeline.py
validation_engine.py

```

```

provenance/
__init__.py
tracker.py
relationship_discovery/
__init__.py
service.py
retrieval/
answer_generation/
__init__.py
base.py
generator.py
citation_validation/
__init__.py
base.py
validator.py
context_builder/
__init__.py
base.py
builder.py
cypher_generation/
__init__.py
base.py
generator.py
schema_mapping.py
schema_patterns.py
schema_utils.py
validator.py
entity_linking/
strategies/
__init__.py
base.py
linker.py
graph_expansion/
__init__.py
base.py
expander.py
graph_retrieval/
__init__.py
base.py
retriever.py
graph_selector/
__init__.py
base.py
service.py
hybrid_retrieval/
__init__.py
base.py
retriever.py
metrics/
__init__.py
base.py
collector.py
ragas_interface.py
ontology_context/
__init__.py
base.py
loader.py
orchestration/
__init__.py
orchestrator.py
query_planning/
__init__.py
base.py
planner.py
query_understanding/
__init__.py
base.py
classifier.py
ranking/
signals/
__init__.py
base.py
ranker.py
schema_introspection/
__init__.py
base.py
introspector.py
vector/
__init__.py
embeddings.py

```

```

entity_store.py
falkordb_search.py
workspace_selector/
__init__.py
base.py
service.py
__init__.py
config.py
formatting.py
models.py
schemas/
__init__.py
constants.py
requests.py
responses.py
services/
__init__.py
analytics_service.py
business_schema_store.py
cypher_service.py
dashboard_service.py
data_dictionary_service.py
export_import_service.py
graph_exploration.py
graph_persistence.py
interfaces.py
normalization_service.py
ontology_orchestrator.py
orchestrator.py
pipeline_service.py
retrieval_service.py
review_service.py
schema_service.py
system_status_service.py
validation_service.py
session/
__init__.py
manager.py
models.py
utils/
__init__.py
profiling.py
serialization.py
validation/
__init__.py
multi_pdf_validator.py
__init__.py
app.py
README.md
settings.py
template/
__init__.py
README.md
unstructured/
agent/
__init__.py
chatbot.py
api/
routes/
__init__.py
demo.py
hitl.py
ingest.py
query.py
__init__.py
main.py
cocoindex_flow/
__init__.py
chunk_fn.py
embed_fn.py
extract_fn.py
flow.py
parse_fn.py
sources.py
structured.py
config/
medical_onto_Medical.yaml
medical_onto_Medical_llmx.yaml
novel_onto_Novel-25646.yaml
novel_onto_Novel-29973.yaml

```

```

novel_onto_Novel-30752.yaml
novel_onto_Novel-40700.yaml
novel_onto_Novel-4128.yaml
novel_onto_Novel-47558.yaml
novel_onto_Novel-54537.yaml
novel_onto_Novel-58553.yaml
novel_ontology.yaml
ontology.example.yaml
ontology.research.yaml
ontology.yaml
schemaorg_medical.yaml
demo/
finance/
autotable.py
chat.py
materialize.py
objects.py
router.py
structured.py
extraction/
__init__.py
baseline.py
clean.py
coref.py
gliner2_model.py
ner.py
relations.py
relations_llm.py
relex.py
run_fresh.py
validate.py
graph/
__init__.py
audit.py
chunk_graph.py
connection.py
context.py
cypher_qa.py
indexes.py
ingest.py
provenance.py
prune.py
query.py
query_rewrite.py
rerank.py
retrieval_substrate.py
run_graph_write.py
schema.py
temporal.py
verify_graph.py
knowledge_object/
__init__.py
builder.py
chunking.py
extraction.py
parser.py
pipeline.py
resolution.py
llm/
__init__.py
embed_cache.py
gateway.py
healthcheck.py
lc.py
list_models.py
memory/
__init__.py
long_term.py
reasoning.py
short_term.py
models/
__init__.py
pydantic_models.py
okf/
__init__.py
PLAN.md
ontology/
__init__.py
_check_onto.py
canonicalize.py

```

```

generate.py
import_schemaorg.py
loader.py
mapper.py
resolver.py
orchestration/
  __init__.py
pipeline.py
state.py
resolution/
  __init__.py
alias_miner.py
analyze_scores.py
blocking.py
linker.py
merge.py
refine.py
run_resolution.py
scoring.py
.llm_usage.json
  __init__.py
chat.py
README.md
  __init__.py
base.py
README.md
llm/
  __init__.py
base.py
client.py
exceptions.py
provider.py
README.md
models/
  __init__.py
canonical.py
domain.py
README.md
normalization/
  __init__.py
base.py
README.md
notebooks/
data/
medicalocr/
cache/
ingested/
needs_review/
pending/
ingested.json
needs_review.json
pending.json
Data/
bills/
discharge_summaries/
output/
ontology/
finance_advisory_ontology_v2
2.owl
telecom-churn (2).owl
workspaces/
workspaces.json
1_Client_Advisory_Transcripts.
pdf
2_Retirement_Policy_Document.p
df
3_Phone_Contact_Workflow.pdf
advisory_sessions 1.csv
churn_risk_profile_enriched
(1).csv
clients 1.csv
enriched_telecom_churn (1).csv
financial_goals 1.csv
interventions_enriched (1).csv
medical.json
novel.json
sample_medical.txt
00_overview.ipynb
01_ingestion_pipeline.ipynb
02_extraction.ipynb

```

```

03_resolution.ipynb
04_retrieval_graphrag.ipynb
05_structured_finance.ipynb
  _build_notebooks.py
knowledge_alignment_lib.py
README.md
requirements.txt
semistrucre.ipynb
structured.ipynb
test.ipynb
test_knowledge_alignment_lib.p
y
ontology/
  __init__.py
base.py
discharge_summary.owl
medical_bill.owl
medical_combined.owl
medicalocr_combined.owl
README.md
semantics.py
pipeline/
alignment/
  __init__.py
knowledge_alignment.py
engines/
  __init__.py
semi_structured_engine.py
structured_engine.py
template_engine.py
unstructured_engine.py
stages/
  __init__.py
alignment.py
base.py
classification.py
cross_doc_link.py
entity_resolution.py
graph_builder.py
indexing.py
ingestion.py
knowledge_object.py
persistence.py
retrieval.py
review.py
semantic_schema.py
validation.py
workspace.py
  __init__.py
context.py
dispatcher.py
execution.py
introspection.py
orchestrator.py
README.md
registry.py
provenance/
  __init__.py
README.md
research/
kg_validation_results.json
README.md
validate_kg.py
retrieval/
  __init__.py
base.py
README.md
scripts/
generate_technical_docx.py
push-dockerhub.ps1
semantic_schema/
  __init__.py
base.py
README.md
services/
  __init__.py
README.md
storage/
  __init__.py
alignment_decision_store.py

```

```

artifact_store.py
base.py
entity_registry_store.py
knowledge_object_store.py
README.md
retrieval_history_store.py
semantic_memory_store.py
temp.py
workspace_store.py
templates/
medical_documents.json
tests/
api/
  __init__.py
test_alignment_api.py
test_ontology_preview.py
test_semi_structured_api.py
config/
test_settings.py
pipeline/
  __init__.py
test_cross_doc_link.py
test_cross_document_identity.p
y
test_graph_builder_relationshi
ps.py
test_input_classifier.py
test_invoice_product_graph.py
test_knowledge_alignment.py
test_knowledge_alignment_stage
.py
test_knowledge_object_store.py
test_langgraph_pipelines.py
test_semantic_memory_store.py
test_semi_structured_business
schema.py
test_semi_structured_engine.py
test_semi_structured_knowledge
_object_builder.py
test_semi_structured_kv_extrac
tion.py
test_semi_structured_templates
.py
test_structured_semantic_fixes
.py
semi_structured/
unit/
test_document_cache_recovery.p
y
test_entity_materializer.py
test_normalize_icd.py
test_ocr.py
test_template_editor.py
test_template_ingest_override.
py
test_template_ocr_noise.py
test_template_storage.py
stabilization/
  __init__.py
test_run_registry.py
test_workspace_stage.py
structured/
contract/
  __init__.py
test_api_contracts.py
cross_domain/
  __init__.py
test_execution_planner_domains
.py
test_healthcare_pipeline.py
test_telecom_pipeline.py
test_university_pipeline.py
e2e/
  __init__.py
test_ontology_csv_e2e.py
fault_injection/
  __init__.py
test_fault_injection.py
integration/
test_session/

```

```

__init__.py
test_multi_doc_ingestion.py
test_session_api.py
__init__.py
test_api.py
test_master_pipeline_ontology.
py
test_ner_pipeline_integration.
py
test_ontology_csv_ingestion.py
test_ontology_integration.py
test_pipeline.py
test_region_based_extraction.p
y
legacy_tests/
__init__.py
test_legacy_extraction.py
test_legacy_ingestion.py
performance/
__init__.py
test_performance.py
regression/
__init__.py
test_ontology_regression.py
test_regression.py
unit/
test_entity_resolution/
__init__.py
test_canonical_service.py
test_cross_document_relationsh
ips.py
test_entity_resolution_service
.py
test_merge_service.py
test_strategies.py
test_session/
__init__.py
test_session_manager.py
test_session_models.py
__init__.py
test_business_execution_schema
.py
test_business_fact_generation.
py
test_business_graph_model.py
test_business_property_persist
ence.py
test_business_schema_store.py
test_business_understanding_se
rvice.py
test_chatbot.py
test_chunking.py
test_client_relationship_deriv
er.py
test_concept_normalizer.py
test_context_window_builder.py
test_cypher_safety.py
test_dataset_analyzer.py
test_demote_concept_endpoint.p
y
test_deterministic_graph_build
er.py
test_domain.py
test_e2e_property_persistence.
py
test_entity_density_analyzer.p
y
test_entity_extractor.py
test_entity_fusion.py
test_entity_generation.py
test_entity_resolution_persist
ence.py
test_execution_planner.py
test_execution_schema.py
test_gateway_client.py
test_gliner_engine.py
test_graph.py
test_graph_analytics.py
test_graph_quality_enhanced.py
test_graph_validation_rules.py

```

```

test_graph_views.py
test_hierarchical_chunking.py
test_ingestion_hardening.py
test_label_resolver.py
test_llm_orchestration_config.
py
test_llm_orchestrator.py
test_master_pipeline.py
test_metrics.py
test_metrics_expanded.py
test_multi_pdf_pipeline.py
test_ner.py
test_ner_config.py
test_ner_hybrid.py
test_nl_to_cypher.py
test_node_builder.py
test_normalizer.py
test_observability.py
test_ontology_class_classifier
.py
test_ontology_cluster_service.
py
test_ontology_context_resolver
.py
test_ontology_discovery.py
test_ontology_domain.py
test_ontology_domain_agnostic.
py
test_ontology_labels.py
test_ontology_mapper.py
test_ontology_orchestrator.py
test_ontology_registry.py
test_ontology_review.py
test_ontology_triple_validator
.py
test_orchestrator.py
test_owl_loader.py
test_parsers.py
test_phase2_apis.py
test_phase2_endpoints.py
test_pii_redactor.py
test_postgres_connector.py
test_postgres_pipeline.py
test_provenance.py
test_provenance_expanded.py
test_query_engine.py
test_rebel_extractor.py
test_relationship_builder.py
test_relationship_discovery.py
test_relationship_extractor.py
test_relationship_generation.p
y
test_relationship_planner.py
test_retrieval_answer_generati
on.py
test_retrieval_citation_valida
tion.py
test_retrieval_context_builder
.py
test_retrieval_cypher_generati
on.py
test_retrieval_entity_linking.
py
test_retrieval_metrics.py
test_retrieval_orchestrator.py
test_retrieval_query_planning.
py
test_retrieval_query_understan
ding.py
test_retrieval_ranking.py
test_retrieval_schema_mapping.
py
test_retrieval_schema_patterns
.py
test_schema_discovery.py
test_schema_introspector.py
test_schema_mapper.py
test_semantic_analyzer.py
test_semantic_enricher.py
test_semantic_graph_persistenc

```

```

e.py
test_semantic_region_builder.p
y
test_semantic_schema.py
test_session_pipeline.py
test_slugify.py
test_type_aware_resolution.py
test_typed_entity_builder.py
test_validation_engine.py
test_workspace_manager.py
conftest.py
unstructured/
__init__.py
conftest.py
smoke.py
test_alias_miner.py
test_canonicalize.py
test_e2e.py
test_gateway_env.py
test_graph.py
test_linker.py
test_linker_graph.py
test_linker_incremental.py
test_mapper.py
test_models.py
test_ner.py
test_ontology.py
test_re.py
test_resolve.py
test_retrieval_substrate.py
test_validate.py
__init__.py
conftest.py
README.md
utils/
__init__.py
datetime_utils.py
hashing.py
README.md
slugify.py
.dockerignore
.env
.env.example
.gitattributes
.gitignore
.python-version
app.py
clear
docker-compose.registry.yml
docker-compose.yml
main.py
pyproject.toml
README.md
uv.lock

```

Figure 5: Directory tree to depth 4



Appendix E — Frontend Module Inventory

Pages and features

- WorkspacePage — workspace CRUD UI
- IngestionPage — file dropzone, ontology preview, scanned tab
- PipelinePage — run progress, SSE stream hook
- KnowledgeObjectsPage — KO table filters
- ReviewPage — concept columns, label bulk actions
- AlignmentPage — semantic memory and override
- GraphPage — CytoscapeCanvas explorer
- RetrievalPage — query modes, Cypher results
- TemplateAuthorWizard, TemplateEditWizard, TemplateGraphEditor, TemplateGraphPreview
- ScanIngestDropzone, ScannedDocumentsPanel
- Dropzone — generic file upload
- DevPanel / DevJson — developer JSON inspector
- WorkspaceHydrator — loads active workspace context

API modules (frontend/src/api/)

- `client.ts` — base fetch + ApiResponse typing
- `workspaces.ts`, `ingestion.ts`, `pipeline.ts`
- `knowledgeObjects.ts`, `review.ts`, `graph.ts`
- `retrieval.ts`, `alignment.ts`, `semiStructured.ts`

Routing

React Router 7 style routes in `frontend/src/routes/index.tsx`; default redirect / → `/workspace`.

Appendix F — Pipeline Stage Source Files

Table 11: Pipeline stage implementation map

Stage	Source file
WorkspaceStage	pipeline/stages/workspace.py
ClassificationStage	pipeline/stages/classification.py
IngestionStage	pipeline/stages/ingestion.py
SemanticSchemaStage	pipeline/stages/semantic_schema.py
KnowledgeObjectStage	pipeline/stages/knowledge_object.py
ReviewStage	pipeline/stages/review.py
EntityResolutionStage	pipeline/stages/entity_resolution.py
KnowledgeAlignmentStage	pipeline/stages/alignment.py
GraphBuilderStage	pipeline/stages/graph_builder.py
PersistenceStage	pipeline/stages/persistence.py
ValidationStage	pipeline/stages/validation.py
IndexingStage	pipeline/stages/indexing.py
RetrievalReadyStage	pipeline/stages/retrieval.py

Engine plugins

Table 12: Engine registry (pipeline/registry.py)

Engine	File	Delegates to
structured	pipeline/engines/structured_engine.py	ingestion/structured parsers + NER
unstructured	pipeline/engines/unstructured_engine.py	ingestion/unstructured/knowledge_object/pipeline.py
template	pipeline/engines/template_engine.py	ingestion/structured/orchestration/langgraph_pipeline.py
semi_structured	pipeline/engines/semi_structured_engine.py	ingestion/semi_structured/orchestration/langgraph_pipeline.py

Appendix G — Structured Parser Classes

Table 13: ingestion/structured/parsers/ (dispatch via `get_parser`)

Parser class	File	Typical extensions
CSVParser	<code>parsers/csv.py</code>	<code>.csv</code>
XLSXParser	<code>parsers/xlsx.py</code>	<code>.xlsx</code> , <code>.xls</code>
JSONParser	<code>parsers/json.py</code>	<code>.json</code>
SQLParser	<code>parsers/sql.py</code>	<code>.sql</code>
PDFParser	<code>parsers/pdf.py</code>	<code>.pdf</code> (structured path)
DOCXParser	<code>parsers/docx.py</code>	<code>.docx</code>
PPTXParser	<code>parsers/pptx.py</code>	<code>.pptx</code>
ImageParser	<code>parsers/image.py</code>	images (OCR cascade)
TextParser	<code>parsers/text.py</code>	<code>.txt</code>
OntologyParser	<code>parsers/ontology.py</code>	OWL/RDF ontology files

Appendix H — Test Suite Areas

Table 14: Pytest layout

Directory	Focus
tests/api/	HTTP contracts — semi-structured, ontology preview
tests/pipeline/	Classifier, graph builder, semi-structured KO builder
tests/structured/unit/	Retrieval stages, NER, ontology, ER, schema, Cypher
tests/structured/integration/	Pipeline and session API integration
tests/structured/cross_domain/	Healthcare, telecom, university sample pipelines
tests/structured/e2e/	Ontology CSV end-to-end
tests/unstructured/	Models, linker, gateway env, validation
tests/semi_structured/unit/	OCR, template editor, OCR noise
tests/config/	Settings loading
tests/stabilization/	Workspace stage, run registry

Appendix I — API Routers and Services Map

Table 15: `api/routers` → `api/services`

Router	Primary services
<code>workspaces.py</code>	<code>workspace_service</code>
<code>ingestion.py</code>	<code>ingestion_service</code>
<code>pipeline.py</code>	<code>pipeline_service</code> , <code>ingestion_service</code> (resume)
<code>knowledge_objects.py</code>	<code>knowledge_object_service</code>
<code>review.py</code>	<code>review_service</code> , <code>concept_service</code>
<code>graph.py</code>	<code>graph_service</code> , <code>workspace_service</code>
<code>retrieval.py</code>	<code>retrieval_service</code>
<code>alignment.py</code>	<code>alignment_service</code>
<code>semi_structured.py</code>	<code>semi_structured_service</code>

Each service module under `api/services/` is the only layer permitted to invoke `PipelineOrchestrator`, `FalkorGraphClient`, and ingestion engines.